



同濟大學
TONGJI UNIVERSITY

操作系统课程设计

xv6 及 Labs 课程项目

实验报告

姓 名： 马小龙

学 号： 2353814

学 院： 计算机科学与技术学院（软件学院）

专 业： 软件工程

指导教师： 王冬青

项目链接： https://github.com/Vestigor/OS_Curriculum_Design.git

二〇二五年七月二十六日

目录

目录	I
实验环境搭建	1
安装 Ubuntu20.04	1
配置 xv6 实验环境	2
建立 github 远程仓库	3
Lab: Xv6 and Unix utilities	5
Boot xv6	5
sleep	6
pingpong	8
Primes	11
Find	14
Xargs	17
提交	20
Lab: system calls	21
System call tracing	21
Sysinfo	25
提交	28
Lab: page tables	29
Speed up system calls	29
Print a page table	31
Detecting which pages have been accessed	35
提交	38
Lab: traps	39
RISC-V assembly	39
Backtrace	42
Alarm	45
提交	49
Lab: Copy-on-Write Fork for xv6	51
Implement copy-on write	51
提交	58
Lab: Multithreading	59
Uthread: switching between threads	59
Using threads	62
Barrier	65
提交	67

Lab: networking	68
Your Job	68
提交	72
Lab: locks	73
Memory allocator	73
Buffer cache	76
提交	82
Lab: file system	83
Large files	83
Symbolic links	88
提交	91
Lab: mmap	92
mmap	92
提交	100

实验环境搭建

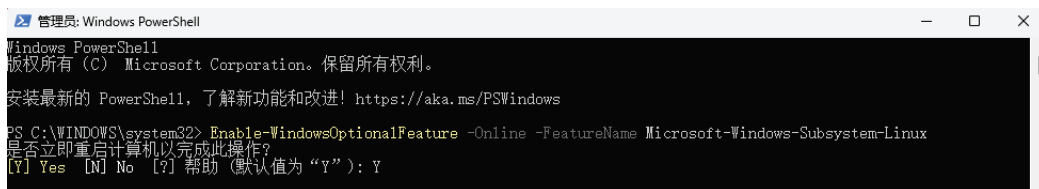
安装 Ubuntu20. 04

Windows 子系统适用于 Linux(Windows Subsystem for Linux,简称 WSL)是 Microsoft 提供的一项功能,允许用户在 Windows 操作系统上运行 Linux 环境,而无需安装虚拟机或双重启动。这对于开发人员和系统管理员非常有用,因为它结合了 Windows 的便利性和 Linux 的强大工具集。安装时使用如下步骤:

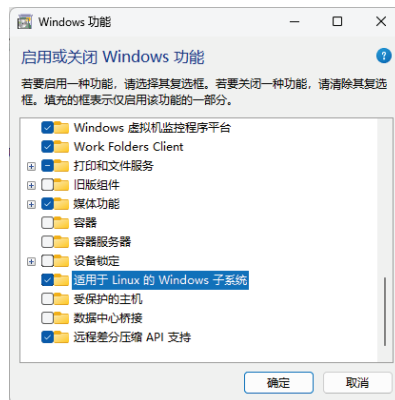
首先我们需要开启个人电脑上的 WSL 支持。打开使用管理员权限的 PowerShell, 输入命令

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

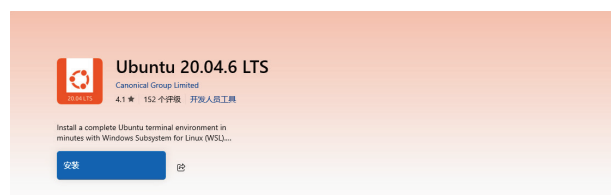
采用以上命令会激活 WSL 服务, 然后需要重启系统。



电脑重启之后, Win + R, 输入 `appwiz.cpl`, 在左上角找到“启动或关闭 Windows 功能”, 此时我们会看到这个选项处于选中状态。表明该服务已经被开启。



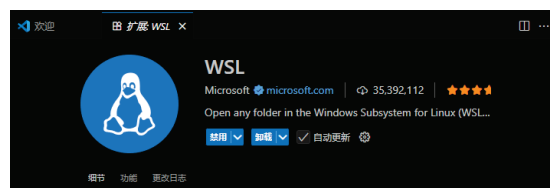
之后在微软应用商店里搜索 Ubuntu 系统, 本次实验选择的是 Ubuntu 20.04 LTS。



下载完后, 打开终端, 根据指导完成用户注册并设定密码。

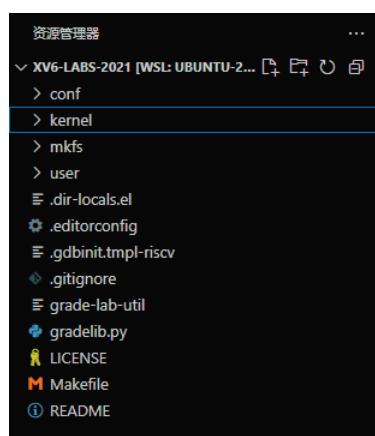
用了 VScode 进行远程 Ubuntu 系统来完成实验，VScode 配置过程如下：

首先在 VSCode 中安装扩展插件 Remote-WSL



在 Ubuntu 终端中切换到 xv6-labs-2021 目录，进入实验文件所在目录，并切换到目标分支，输入 `code .` 命令，即可在当前目录打开 Windows 下的 VSCode，进行相关实验。

```
$ cd xv6-labs-2021
$ git checkout <分支名> 例如: git checkout util
$ code .
```



配置 VScode 后，可以在资源管理器界面右键打开集成终端来执行终端命令，以后便可以直接通过 VScode 来进实验。

建立 github 远程仓库

首先需要在 github 上建立一个远端仓库，同时获取该远端仓库的 HTTPS。我的仓库地址为：

```
https://github.com/Vestigor/OS_Curriculum_Design.git
```

接着在 WSL 终端打开实验文件所在的子目录 xv6-labs-2021。

```
$ cd xv6-labs-2021
```

使用以下命令为该项目添加 git 远程仓库地址。

```
$ git remote add github https://github.com/Vestigor/OS_Curriculum_Design.git
```

之后便可以使用以下命令将完成的分支推送到 github，完成对应实验提交。

```
$ git checkout <分支名>
```

```
$ git push github <分支名>:<分支名>
```

Lab: Xv6 and Unix utilities

Boot xv6

[实验目的]

利用 WSL 终端切换到 xv6-labs-2020 代码的 util 分支，同时实验 qemu 模拟器启动并运行 xv6 系统，观察实验现象和结果。

[实验内容]

获取实验室的 xv6 源代码并签出 util 分支:

```
$ git clone git://g.csail.mit.edu/xv6-labs-2021
$ cd xv6-labs-2021
$ git checkout util
```

在 xv6-labs-2021 目录下的终端中输入 `make qemu` 来启动 xv6

\$ make qemu

```
问题    输出    调试控制台    终端    窗口
```

```
• bruce@jarvis:/xv6-labs-2021$ git checkout util  
Switched to branch 'util'  
Your branch is up to date with 'origin/util'.  
  
• bruce@jarvis:/xv6-labs-2021$ make qemu  
riscv64-linux-gnu-gcc      -c   o kernel/entry.o kernel/entry.S  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/initmain.o kernel/initmain.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/main.o kernel/main.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/printk.o kernel/printk.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/string.o kernel/string.c  
riscv64-linux-gnu-gcc      -c   o kernel/switch.o kernel/switch.S  
riscv64-linux-gnu-gcc      -c   o kernel/trampoline.o kernel/trampoline.S  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/user.o kernel/user.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/vfs.o kernel/vfs.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/warrior.o kernel/warrior.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/x86_64.o kernel/x86_64.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/zlib.o kernel/zlib.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/zlib_stub.o kernel/zlib_stub.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/zlib_test.o kernel/zlib_test.c  
riscv64-linux-gnu-gcc      -Wall -Werror -o -fno-omit-frame-pointer -ggdb -DSOL_UTIL kernel/zlib_test.o kernel/zlib_test.c
```

xv6 启动成功后我们可以在终端中输入 `ls`，显示出来的是 `mkfs` 包含的初始文件系统。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2226
xargstest.sh 3 3 93
cat        2 4 23896
echo       2 6 22728
forktest   2 6 13080
grep       2 7 27248
init       2 8 23824
kill       2 9 22696
ln         2 10 22648
ls         2 11 26128
mkdir     2 12 22800
rm         2 13 22792
sh         2 14 41664
stressfs   2 15 23800
usertests  2 16 156008
grind      2 17 37968
wc         2 18 25040
zombie    2 19 22192
console    3 20 0
$ []
```


如果要退出 `qemu`，需要键入 `Ctrl-a x`，即先按下 `Ctrl-a`，松开后再按下 `x`，就可退出 `qemu`。

[实验中遇到的问题和解决方法]

该实验的内容较为简单，我并没有遇到什么问题。

[实验心得]

本次实验围绕获取 `xv6` 源代码并启动系统展开，让我在知识和理论层面有了不少收获。

通过执行 `git clone` 获取源代码、`git checkout util` 切换分支的操作，我对版本控制工具在项目管理中的作用有了更实际的认识。这让我明白，对于像 `xv6` 这样的操作系统项目，不同分支可能对应不同的实验内容或功能版本，使用 `Git` 可以便捷地在不同版本间切换，为开发和学习提供了灵活的环境，也加深了我对“分支管理是大型项目协作基础”这一理论的理解。

启动 `xv6` 的过程让我直观感受到了操作系统从代码到可运行状态的转化。输入 `make qemu` 后，系统经历编译、加载等环节，最终成功启动。而在启动后输入 `ls` 命令查看初始文件系统，让我对该操作系统的文件管理机制有了初步认知，了解到 `mkfs` 构建的初始文件系统是用户与系统交互的基础之一，文件的组织和展示背后是文件系统对存储结构的管理和解析。

该实验非常基础，是后续所有实验的必要内容，这次实验为后续的实验打下了基础。

sleep

[实验目的]

Implement the UNIX program `sleep` for `xv6`; your `sleep` should pause for a user-specified number of ticks. A tick is a notion of time defined by the `xv6` kernel, namely the time between two interrupts from the timer chip. Your solution should be in the file `user/sleep.c`.

本实验是为 `xv6` 实现 UNIX 的 `sleep` 程序，你的 `sleep` 应能按照用户指定的 tick 数暂停执行。其中，tick 是 `xv6` 内核定义的时间单位，即定时器芯片两次中断之间的时间间隔。你的实现代码应位于文件 `user/sleep.c` 中。

[实验内容]

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

// argc 是命令行总的参数个数
// argv[] 是 argc 个参数，其中第 0 个参数是程序的全名，之后的参数是命令行后面跟的用户输入的参数
int main(int argc, char *argv[]) {
    // 参数个数不为 2，说明命令格式错误
    if (argc != 2) {
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    int ticks = atoi(argv[1]);

    // 不能睡眠小于 0 的时间
    if (ticks < 0) {
        fprintf(2, "sleep: invalid time\n");
        exit(1);
    }

    sleep(ticks);
    exit(0);
}
```

该程序的核心功能是通过系统调用让进程暂停指定的 ticks 时长。

在 C 语言中，int argc, char *argv[] 是 main 函数的标准参数，用于接收命令行输入参数。argc 为整数，代表命令行参数的数量（包含程序本身名字，故至少为 1）；argv 是字符指针数组，每个元素为一个字符串，argv[0] 是程序名，argv[1] 及以后为命令行参数。

main 函数中，首要工作是校验命令行参数。程序需以“程序名 + 睡眠时长”的格式调用，因此通过判断 argc != 2 检查参数数量是否符合要求。若不符，用 fprintf(2, ...) 向标准错误输出提示信息“usage: sleep”，再调用 exit(1) 终止程序。

接着，用 atoi(argv[1]) 将 argv[1] 所指向的字符串类型时长转为整数 ticks，随后判断 ticks < 0，若为真则输出“sleep: invalid time”并退出，以此确保睡眠时长为非负数，完成对输入参数有效性的校验。

最终，调用 sleep(ticks) 系统调用使进程进入睡眠状态，待指定 ticks 后唤醒，再通过 exit(0) 正常结束程序。

完成编写后，在 Makefile 文件中 UPROGS 添加一行 \$U/_sleep\ 配置，然后

进行程序测试。

在 xv6 的 qemu 环境中运行 sleep 10 命令，观察到进程停顿现象。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$
```

为该实验运行成绩测试：

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util sleep
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (0.5s)
== Test sleep, returns == sleep, returns: OK (0.9s)
== Test sleep, makes syscall == sleep, makes syscall: OK (1.0s)
```

[实验中遇到的问题和解决方法]

该实验的内容较为简单，我并没有遇到什么问题。

[实验心得]

通过这个实验，我了解了如何在 xv6 操作系统中实现一个基于系统调用的简单程序，掌握了程序从参数处理到调用系统调用完成功能的基本流程，包括错误处理时输出提示信息 and 设置合适退出状态码的方法；其次，通过对 argc 和 argv 的运用与理解，我学会了如何处理命令行参数，能够编写可接收用户输入并据此调整行为的命令程序，同时也清楚了 atoi 等辅助函数在参数处理中的作用，加深了我对用户态程序与内核交互的认识。

pingpong

[实验目的]

Write a program that uses UNIX system calls to "ping-pong" a byte between two processes over a pair of pipes, one for each direction. The parent should send a byte to the child; the child should print "<pid>: received ping", where <pid> is its process ID, write the byte on the pipe to the parent, and exit; the parent should read the byte from the child, print "<pid>: received pong", and exit. Your solution should be in the file user/pingpong.c.

编写一个使用 UNIX 系统调用的程序，通过一对管道（每个方向一个）在两

个进程之间"乒乓"传递一个字节。父进程应向子进程发送一个字节；子进程应打印"<pid>: received ping"，其中<pid>是其进程 ID，然后将该字节写入到父进程的管道后退出；父进程应从子进程读取该字节，打印"<pid>: received pong"，然后退出。你的解决方案应放在文件 user/pingpong.c 中。

[实验内容]

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
    int p2c[2]; // pipe: parent to child
    int c2p[2]; // pipe: child to parent
    pipe(p2c);
    pipe(c2p);
    int pid = fork();
    if (pid < 0) {
        fprintf(2, "fork failed\n");
        exit(1);
    }
    if (pid == 0) {
        // 子进程
        close(p2c[1]); // 子进程不从 p2c 写
        close(c2p[0]); // 子进程不从 c2p 读
        char byte;
        read(p2c[0], &byte, 1); // 从父进程读取一个字节
        printf("%d: received ping\n", getpid());
        write(c2p[1], &byte, 1); // 写回给父进程
        // 管道使用完毕后，释放剩余端
        close(p2c[0]);
        close(c2p[1]);
    } else {
        // 父进程
        close(p2c[0]); // 父进程不从 p2c 读
        close(c2p[1]); // 父进程不从 c2p 写
        char byte = 'A';
        write(p2c[1], &byte, 1); // 发一个字节给子进程
        read(c2p[0], &byte, 1); // 读取回来的字节
        printf("%d: received pong\n", getpid());
        // 管道使用完毕后，释放剩余端口
        close(p2c[1]);
        close(c2p[0]);
        wait(0); // 等待子进程退出
    }
}
```

```

    }
    exit(0);
}

```

该程序的核心功能是通过两个管道实现父子进程间的双向通信。首先创建两个管道 p2c（父到子）和 c2p（子到父），使用 fork 创建子进程后，父子进程分别关闭不需要的管道端（避免读写混乱）。子进程从 p2c 读取字节后，打印带自身 PID 的 "received ping"，再通过 c2p 回传字节，释放剩余管道端并结束进程；父进程向 p2c 发送字节，待读取 c2p 传回的字节后，打印带自身 PID 的 "received pong"，释放剩余管道端并通过 wait 等待子进程结束后结束。

fork 是 Unix 系统调用，它用于创建一个新的进程，新的进程是调用进程的副本。子进程执行 fork() 时返回 0。父进程执行 fork() 时返回新创建的子进程的进程 ID (PID)。

完成编写后，在 Makefile 的 UPROGS 中添加一行 \$U/_pingpong\ 配置。

在 xv6 的 qemu 环境中运行 pingpong 命令，可观察到父子进程分别打印对应的信息。

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ pingpong
4: received ping
3: received pong
$

```

为该实验运行成绩测试：

```

● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util pingpong
make: 'kernel/kernel' is up to date.
== Test pingpong == pingpong: OK (0.9s)
(Old xv6.out.pingpong failure log removed)

```

[实验中遇到的问题和解决方法]

初始编写时，我对 fork 的返回值理解模糊，因此并不清楚如何区分父子进程。查阅资料后明确，父进程调用 fork 后会获得子进程的 PID（大于 0），而子进程则返回 0。通过判断 pid 的值（pid==0 为子进程，pid>0 为父进程），将两者的读写逻辑分离，分别编写对应代码，最终实现正确的通信流程。

[实验心得]

通过本次实验，我深入理解了 fork 创建进程的机制，明确返回值在父子进程区分中的关键作用，掌握了进程独立运行与逻辑分离的实现方法。其次，我清

晰认识到管道的单向通信特性，学会了通过成对管道构建双向通信通道，理解关闭闲置端口对避免阻塞的重要性。最后，我对 `wait` 系统调用的作用有了具体认知，体会到进程同步在多进程交互中的必要性，整体加深了对 UNIX 系统调用协同工作的理解。

Primes

[实验目的]

Write a concurrent version of prime sieve using pipes. This idea is due to Doug McIlroy, inventor of Unix pipes. The picture halfway down this page and the surrounding text explain how to do it. Your solution should be in the file `user/primes.c`.

使用管道实现一个并发的素数筛程序。这个思路源自 Unix 管道的发明者 Doug McIlroy。本页中间的图示及周围文字说明了如何实现。你的解决方案应放在文件 `user/primes.c` 中。

[实验内容]

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void subProcess(int pfd[2]) {
    close(pfd[1]); // 只读
    int prime;
    if (read(pfd[0], &prime, sizeof(int)) == 0) {
        close(pfd[0]);
        exit(0);
    }
    printf("prime %d\n", prime);
    int n;
    int newp[2];
    pipe(newp);
    if (fork() == 0) {
        // 子进程递归处理
        close(newp[1]);
        subProcess(newp);
    } else {
        close(newp[0]); // 父只写
        while (read(pfd[0], &n, sizeof(int)) > 0) {
```

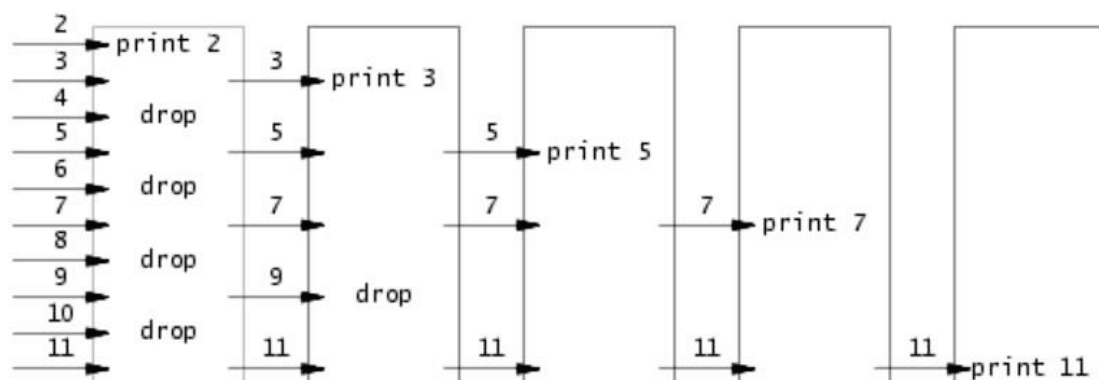
```

        if (n % prime != 0) {
            write(newp[1], &n, sizeof(int));
        }
    }
    close(pfd[0]);
    close(newp[1]);
    wait(0);
    exit(0);
}
}

int main() {
    int p[2];
    pipe(p);
    if (fork() == 0) {
        // 子进程处理筛选
        subProcess(p);
        exit(0);
    }
    else {
        // 父进程写入 2~35
        close(p[0]); // 父写
        for (int i = 2; i <= 35; i++) {
            write(p[1], &i, sizeof(int));
        }
        close(p[1]);
        wait(0);
        exit(0);
    }
}

```

该实验代码基于以下图示流程：



该程序的核心功能是通过管道和进程创建实现并发质数筛选。main 函数创建管道后，通过 fork 生成子进程：父进程负责向管道写入 2 到 35 的整数，写完后关闭写端并等待子进程结束；子进程调用 subProcess 函数处理筛选逻辑。

`subProcess` 函数中，进程先从管道读取第一个数作为质数并打印，然后创建新管道和子进程。当前进程（父）读取剩余数字，将不能被该质数整除的数写入新管道；新子进程递归调用 `subProcess` 处理新管道中的数字，直到管道读取完毕（`read` 返回 0），释放资源并退出。

`fork` 用于创建新进程，子进程返回 0，父进程返回子进程 PID，以此实现递归创建筛选进程链；`pipe` 创建单向通信管道，实现数字在进程间的传递。

完成编写后，在 `Makefile` 的 `UPROGS` 中添加 `$U/_primes\` 配置。

在 `xv6` 的 `qemu` 环境中运行 `primes` 命令，可观察到父子进程分别打印对应的信息。

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

为该实验运行成绩测试：

```
bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util primes
make: 'kernel/kernel' is up to date.
== Test primes == primes: OK (0.8s)
```

[实验中遇到的问题和解决方法]

开始是，我并不知道清除 `read` 的返回值机制，所以并不知道什么时候是子进程读取管道结束的标志，什么时候该停止读、关闭读端，因而并未对该部分进行处理，产生了即使父进程已经不在写入时，子进程始终在读取管道内容，进而导致程序出错。后续我在 B 站找到了相关教程，了解了当父进程关闭写端时，子进程 `read` 会返回 0，可以依据该内容关闭读端，然后写入新管道完毕后，就可以等待该进程的子进程结束后结束。

[实验心得]

通过本次实验，我深入理解了管道与 `fork` 结合实现并发筛选的机制，掌握了递归创建进程链的设计思路。同时，我清晰认识到关闭闲置文件描述符对资源管理的重要性，学会了通过 `read` 返回值判断管道状态以及及时做出响应从而避免阻塞。最后，我对进程间数据传递的顺序性和独立性有了具体认知，体会到 UNIX

系统调用在构建复杂并发程序中的协同作用，加深了对 "每个进程专注单一筛选任务" 的分工思想的理解。

Find

[实验目的]

Write a simple version of the UNIX find program: find all the files in a directory tree with a specific name. Your solution should be in the file user/find.c.

编写一个 UNIX find 程序的简化版本：在目录树中查找具有特定名称的所有文件。你的解决方案应放在文件 user/find.c 中。

[实验内容]

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

char *fmtname(char *path)
{
    char *p;
    // Find first character after last slash.
    for (p = path + strlen(path); p >= path && *p != '/'; p--)
        ;
    p++;
    return p;
}

void find(char *path, char *target)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;
    if ((fd = open(path, 0)) < 0)
    {
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }
    if (fstat(fd, &st) < 0)
    {
        fprintf(2, "find: cannot stat %s\n", path);
    }
}
```

```

        close(fd);
        return;
    }
    switch (st.type)
    {
    case T_FILE:
        if (strcmp(fmtname(path), target) == 0)
            printf("%s\n", path);
        break;
    case T_DIR:
        if (strlen(path) + 1 + DIRSIZ + 1 > sizeof buf)
        {
            printf("find: path too long\n");
            break;
        }
        strcpy(buf, path);
        p = buf + strlen(buf);
        *p++ = '/';
        while (read(fd, &de, sizeof(de)) == sizeof(de))
        {
            if (de.inum == 0)
                continue;
            // 跳过 "." 和 ".."
            if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
                continue;
            memmove(p, de.name, DIRSIZ);
            p[DIRSIZ] = 0;
            if (stat(buf, &st) < 0)
            {
                printf("find: cannot stat %s\n", buf);
                continue;
            }
            // 递归调用
            find(buf, target);
        }
        break;
    }
    close(fd);
}

int main(int argc, char *argv[])
{
    if (argc < 3)
    {
        fprintf(2, "find: not enough arguments\n");
    }
}

```

```

        exit(1);
    }
    find(argv[1], argv[2]);
    exit(0);
}

```

该程序通过递归遍历目录树，实现查找指定名称文件的功能，核心代码围绕路径处理与目录遍历展开。程序首先定义了 `fmtname` 函数，其作用是从完整路径中提取文件名：通过从路径字符串的末尾向前遍历，找到最后一个斜杠的位置，返回斜杠之后的部分，例如将 `"/a/b"` 转换为 `"b"`，这为后续文件名的比对提供了关键支持。

`find` 函数是实现查找功能的核心，采用递归方式处理文件和目录。它首先尝试打开目标路径，然后通过 `fstat` 系统调用获取路径对应的文件状态，以此区分当前对象是文件还是目录。当处理的是文件时，调用 `fmtname` 函数提取文件名，与目标名称进行比对，一致时说明找到了一个对应项目，打印该文件的完整路径。当处理的是目录时，程序先将原路径复制到缓冲区，在末尾添加斜杠，再将目录项中的子名称复制到斜杠之后，以此拼接形成子目录的路径。在读取目录项的过程中，程序会跳过 `"."` 和 `".."` 这两个特殊目录，避免递归过程中出现循环遍历的问题。对于每个有效的子目录项，`find` 函数会递归调用自身，继续深入查找。

`main` 函数则负责参数校验，确保用户提供了起始目录和目标文件名两个必要参数，并调用 `find` 函数开始查找过程。

完成代码编写后，在 `Makefile` 的 `UPROGS` 中添加 `$U/_find\` 配置。

在 `xv6` 的 `qemu` 环境中运行，通过依次执行以下指令来测试程序。

```

$ echo > b
$ mkdir a
$ echo > a/b
$ find . b

```

测试结果如下：

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
$

```

为该实验运行成绩测试：

```

● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util find
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (1.4s)
(Old xv6.out.find_curdir failure log removed)
== Test find, recursive == find, recursive: OK (1.0s)
(Old xv6.out.find_recursive failure log removed)

```

[实验中遇到的问题和解决方法]

该实验如果完全独立完成较为困难，但根据教程提示，只需对 `ls.c` 进行部分改写便可以完成，实际做起来并不困难，未遇到难题。

[实验心得]

本次实验让我对文件系统的层次结构有了更直观的认识。通过递归遍历目录树的过程，我清晰地看到了目录与文件如何组织成一个树形结构，而递归方法恰好能高效地处理这种嵌套结构，逐层深入每个子目录完成查找任务。

同时，实验也加深了我对 UNIX 系统中 "一切皆文件" 理念的理解。目录作为一种特殊的文件，其内容是包含子目录项的结构体，通过 `open`、`read` 等系统调用可以像操作普通文件一样访问目录内容，这种设计使得文件系统的交互逻辑更加统一。而 `stat`、`fstat` 等系统调用则为获取文件属性、区分文件类型提供了关键支持，是实现目录遍历和文件查找的基础。

最后，递归思想在本次实验中的应用让我受益匪浅。面对目录树这种复杂的嵌套结构，递归能够将问题分解为多个相同的子问题，每个子问题只需处理当前目录的内容，大大简化了代码逻辑。这种分而治之的思路，不仅适用于目录查找，在处理其他具有层次结构的问题时也同样有效，为我今后解决类似问题提供了重要启发。

Xargs

[实验目的]

Write a simple version of the UNIX `xargs` program: read lines from the standard input and run a command for each line, supplying the line as arguments to the command. Your solution should be in the file `user/xargs.c`.

编写一个简单版本的 UNIX `xargs` 程序：从标准输入读取行，为每行运行一个命令，并将该行作为参数提供给该命令。解决方案应位于文件 `user/xargs.c` 中。

[实验内容]

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
```

```

#include "kernel/param.h"
int main(int argc, char *argv[])
{
    char buf[512];
    int i, n;
    char *p = buf;
    char *args[MAXARG];
    // 拷贝初始命令参数
    for (i = 0; i < argc - 1 && i < MAXARG - 1; i++)
    {
        args[i] = argv[i + 1];
    }
    // 每次读取标准输入的一个字符
    while ((n = read(0, p, 1)) > 0)
    {
        // 每次读取一行后开始处理
        if (*p == '\n')
        {
            *p = '\0';    // 将换行符变为字符串结束符
            args[i] = buf; // 添加当前行作为最后一个参数
            args[i + 1] = 0;
            // 创建子进程
            if (fork() == 0)
            {
                exec(args[0], args);
                fprintf(2, "exec failed\n");
                exit(1);
            }
            else
            {
                wait(0);
            }
            // 重置读取 buffer
            p = buf;
        }
        else
        {
            p++;
        }
    }
    exit(0);
}

```

该程序的核心功能是读取标准输入中的每行内容，将其作为参数传递给指定命令并执行，实现了输入与命令的动态结合。

首先，程序处理初始命令参数。`main` 函数接收的 `argv` 包含用户指定的基础命令，程序将这些参数拷贝到 `args` 数组中，作为后续执行命令的基础部分。这里利用了 `kernel/param.h` 中定义的 `MAXARG` 来限制参数数量，确保不超过系统允许的最大参数个数。

接着，程序读取标准输入。通过循环每次读取一个字符到缓冲区 `buf`，直到遇到换行符 `\n`，表示一行输入结束。此时将换行符替换为字符串终止符 `\0`，使该行内容成为可作为参数的字符串，并将其添加到 `args` 数组的末尾，形成完整的命令参数列表。

最后，程序会创建子进程执行命令。通过 `fork` 创建子进程，在子进程中调用 `exec` 执行 `args` 数组指定的命令；父进程则通过 `wait` 等待子进程完成，确保命令按顺序执行。每次处理完一行输入后，重置缓冲区指针，准备读取下一行，直到标准输入结束。

完成代码编写后，在 `Makefile` 的 `UPROGS` 中添加 `$U/_xargs\` 配置。

即可在 `xv6` 环境中运行。例如执行 `echo hello too | xargs echo bye`，程序会输出 `bye hello too`，验证功能正确性。

在 `xv6` 的 `qemu` 环境中运行 `echo hello too | xargs echo bye` 命令，测试程序正确性。

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo hello too | xargs echo bye
bye hello too
$
```

为该实验运行成绩测试：

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util xargs
make: 'kernel/kernel' is up to date.
== Test xargs == xargs: OK (1.2s)
```

[实验中遇到的问题和解决方法]

该程序实现较为简单，关键在于理解 `xargs` 命令的用途。看到实验内容时，我并没有很好的理解 `xargs` 命令是用来做什么的，查看教程后才知道 `xargs` 命令其实是作为标准输入与其他命令之间的桥梁，它从标准输入读取内容（按行分割），将每行内容作为参数传递给指定的命令，以便实现对该命令的执行。了解起作用后，编写代码便不再困难。

[实验心得]

本次实验通过读取标准输入并动态构建命令参数,我理解了 `xargs` 作为“输入转参数”桥梁的作用,尤其是如何将管道传来的内容转化为命令的一部分。

此外,通过对 `fork`、`exec` 与 `wait` 的使用,我对他们协同工作机制有了更深入的理解。父进程等待子进程的设计确保了命令执行的顺序性,而子进程通过 `exec` 加载新程序的过程,让我直观感受到进程替换的特性,进一步理解了 UNIX 中“命令即进程”的设计思想。

提交

首先创建一个 `time.txt` 文件,在其中存放一个整数,记录实验的小时数。

然后执行命令 `./grade-lab-util`, 获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-util
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (1.6s)
== Test sleep, returns == sleep, returns: OK (0.7s)
== Test sleep, makes syscall == sleep, makes syscall: OK (0.8s)
== Test pingpong == pingpong: OK (1.2s)
== Test primes == primes: OK (0.8s)
== Test find, in current directory == find, in current directory: OK (1.1s)
== Test find, recursive == find, recursive: OK (1.1s)
== Test xargs == xargs: OK (1.1s)
== Test time ==
time: OK
Score: 100/100
```

最后在当前分支 (util) 下,依次执行以下命令,提交分支到 github:

```
$ git checkout util    #确保在 util 分支
$ make clean           #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: Xv6 and Unix utilities."
$ git push github util:util
```

Lab: system calls

实验开始前，依次执行以下命令，切换到 syscall 分支

```
$ git fetch
$ git checkout syscall
$ make clean
```

System call tracing

[实验目的]

In this assignment you will add a system call tracing feature that may help you when debugging later labs. You'll create a new trace system call that will control tracing. It should take one argument, an integer "mask", whose bits specify which system calls to trace. For example, to trace the fork system call, a program calls `trace(1 << SYS_fork)`, where `SYS_fork` is a syscall number from `kernel/syscall.h`. You have to modify the xv6 kernel to print out a line when each system call is about to return, if the system call's number is set in the mask. The line should contain the process id, the name of the system call and the return value; you don't need to print the system call arguments. The trace system call should enable tracing for the process that calls it and any children that it subsequently forks, but should not affect other processes.

在此作业中，您将添加一个系统调用跟踪功能，该功能可能在您调试后续实验时提供帮助。您需要创建一个新的 `trace` 系统调用来控制跟踪行为。该系统调用应接受一个参数：一个整数“掩码”（mask），其二进制位指定了需要跟踪哪些系统调用。

例如，要跟踪 `fork` 系统调用，一个程序应调用 `trace(1 << SYS_fork)`，其中 `SYS_fork` 是来自 `kernel/syscall.h` 的系统调用编号。

您需要修改 xv6 内核，使得在每一个系统调用即将返回时，如果该系统调用的编号在掩码中被设置，则打印一行信息。该行信息应包含：进程 ID (process id)、系统调用的名称以及返回值；您不需要打印系统调用的参数。

`trace` 系统调用应当为调用它的进程以及它后续通过 `fork` 创建的任何子进程启用跟踪功能，但不应影响其他进程。

[实验内容]

根据教程内容，首先在在 Makefile 中向 UPROGS 添加 一行以下代码

```
$U/_trace\
```

当前系统调用的用户空间存根还不存在，因此首先需要添加 trace 用户态接口

1.将系统调用的原型添加到 user/user.h,以便让用户程序能识别并调用 trace() 函数

```
int trace(int);
```

2.将存根添加到 user/usys.pl, 该脚本会自动生成封装 trace 的系统调用包装函数，并将它写入 usys.S 文件，用 ecall 指令完成从用户态到内核的跳转，完成后，就可以通过通过 trace() 调用内核函数。

```
entry("trace");
```

3.将系统调用编号添加到 kernel/syscall.h, 22 是通常用于 trace 的默认编号，实际值不能与已有编号冲突。

```
#define SYS_trace 22
```

接下来在内核中实现 trace 系统调用。

1.需要在 kernel/proc.h 文件的 proc 结构体中加入一个新的变量，让每个进程都有一个自己的掩码。

```
int tracemask; // 每个进程的追踪掩码
```

2.在 kernel/sysproc.c 添加 sys_trace 实现：

```
uint64
sys_trace(void) {
    int mask;
    if (argint(0, &mask) < 0)
        return -1;
    myproc()->tracemask = mask;
    return 0;
}
```

然后，修改 fork(), 使得子进程继承父进程的 trace 掩码，这需要在 kernel/proc.c 中 fork() 实现部分添加：

```
np->tracemask = p->tracemask; // 子进程继承父进程的追踪掩码
```

最后，需要修改 syscall() 实现打印追踪信息

1.添加 sys_trace(void)函数声明

```
extern uint64 sys_trace(void);
```

2.补全函数指针数组 static uint64 (*syscalls[])(void), 添加

```
[SYS_trace] sys_trace,
```

3.添加系统调用名数组

```
static char *syscall_names[] = {
    [SYS_fork]   "fork",
    [SYS_exit]   "exit",
    [SYS_wait]   "wait",
    [SYS_pipe]   "pipe",
    [SYS_read]   "read",
    [SYS_kill]   "kill",
    [SYS_exec]   "exec",
    [SYS_fstat]  "fstat",
    [SYS_chdir]  "chdir",
    [SYS_dup]    "dup",
    [SYS_getpid] "getpid",
    [SYS_sbrk]   "sbrk",
    [SYS_sleep]  "sleep",
    [SYS_uptime] "uptime",
    [SYS_open]   "open",
    [SYS_write]  "write",
    [SYS_mknod]  "mknod",
    [SYS_unlink] "unlink",
    [SYS_link]   "link",
    [SYS_mkdir]  "mkdir",
    [SYS_close]  "close",
    [SYS_trace]  "trace",
};
```

4. 修改 syscall() 函数，在系统调用返回前添加打印信息

```
void
syscall(void)
{
    int num;
    struct proc *p = myproc();
    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->a0 = syscalls[num]();
        if (p->tracemask & (1 << num)) {
            printf("%d: syscall %s -> %d\n", p->pid,
                syscall_names[num] ? syscall_names[num] : "unknown",
                p->trapframe->a0);
        }
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}
```

完成代码编写后，在 xv6 的 qemu 环境以下中运行如下测试命令，观察到测试全部通过。

```
$ trace 32 grep hello README
$ trace 2147483647 grep hello README
$ grep hello README
$ trace 2 usertests forkforkfork
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 968
3: syscall read -> 235
3: syscall read -> 0
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 968
4: syscall read -> 235
4: syscall read -> 0
4: syscall close -> 0
$ grep hello README
```

```
$ trace 2 usertests forkforkfork
usertests starting
6: syscall fork -> 7
test forkforkfork: 6: syscall fork -> 8
8: syscall fork -> 9
9: syscall fork -> 10
9: syscall fork -> 11
10: syscall fork -> 12
11: syscall fork -> 13
9: syscall fork -> 14
10: syscall fork -> 15
12: syscall fork -> 16
11: syscall fork -> 17
9: syscall fork -> 18
12: syscall fork -> 19
11: syscall fork -> 20
9: syscall fork -> 21
10: syscall fork -> 22
11: syscall fork -> 23
12: syscall fork -> 24
9: syscall fork -> 25
10: syscall fork -> 26
11: syscall fork -> 27
13: syscall fork -> 28
9: syscall fork -> 29
25: syscall fork -> 30
10: syscall fork -> 31
9: syscall fork -> 32
10: syscall fork -> 33
11: syscall fork -> 34
9: syscall fork -> 35
12: syscall fork -> 36
10: syscall fork -> 37
9: syscall fork -> 38
14: syscall fork -> 39
10: syscall fork -> 40
13: syscall fork -> 41
9: syscall fork -> 42
11: syscall fork -> 43
10: syscall fork -> 44
12: syscall fork -> 45
11: syscall fork -> 46
9: syscall fork -> 47
10: syscall fork -> 48
11: syscall fork -> 49
20: syscall fork -> 50
9: syscall fork -> 51
11: syscall fork -> 52
10: syscall fork -> 53
9: syscall fork -> 54
13: syscall fork -> 55
10: syscall fork -> 56
9: syscall fork -> 57
12: syscall fork -> 58
11: syscall fork -> 59
9: syscall fork -> 60
10: syscall fork -> 61
11: syscall fork -> 62
12: syscall fork -> 63
10: syscall fork -> 64
11: syscall fork -> 65
12: syscall fork -> 66
10: syscall fork -> 67
9: syscall fork -> 68
25: syscall fork -> -1
10: syscall fork -> -1
9: syscall fork -> -1
11: syscall fork -> -1
50: syscall fork -> -1
OK
6: syscall fork -> 69
ALL TESTS PASSED
```

[实验中遇到的问题和解决方法]

这次实验是第一次完整地实现一次系统调用，主要是在指导的参考下和参考其他系统调用的实现过程来实现系统调用跟踪，总体来说不是很难。

[实验心得]

这次实验是一次很好的实践机会，让我对操作系统内核有了更深入的理解。通过实现 `trace` 系统调用，我不仅掌握了系统调用的实现方法，还学会了如何在大型代码库中添加新功能。这对我后续的学习和实验非常有帮助。

Sysinfo

[实验目的]

In this assignment you will add a system call, `sysinfo`, that collects information about the running system. The system call takes one argument: a pointer to a struct `sysinfo` (see `kernel/sysinfo.h`). The kernel should fill out the fields of this struct: the `freemem` field should be set to the number of bytes of free memory, and the `nproc` field should be set to the number of processes whose state is not `UNUSED`. We provide a test program `sysinfotest`; you pass this assignment if it prints "`sysinfotest: OK`".

在本实验中，你将添加一个名为 `sysinfo` 的系统调用，用于收集运行系统的信息。该系统调用接收一个参数：指向 `struct sysinfo` 的指针（定义见 `kernel/sysinfo.h`）。内核需填充该结构体的字段：`freemem` 字段应设置为空闲内存的字节数，`nproc` 字段应设置为状态非 `UNUSED` 的进程数量。我们提供了测试程序 `sysinfotest`；当该程序输出 "`sysinfotest: OK`" 时，即表示你已通过本次实验。

[实验内容]

在 `Makefile` 中添加测试程序 `sysinfotest`，让 `make qemu` 时将用户程序 `user/sysinfotest.c` 编译并链接成 `_sysinfotest`，使其可以在 `xv6` 中运行。找到 `UPROGS` 项，添加：

```
$U/_sysinfotest\
```

在 `user/user.h` 声明 `struct sysinfo` 结构体和 `sysinfo` 用户态调用接口，让 `user/sysinfotest.c` 编译器知道函数和结构体的存在，防止编译错误。

```
struct sysinfo;
int sysinfo(struct sysinfo *);
```

在 `user/usys.pl` 中添加一行存根

```
entry("sysinfo");
```

将系统调用编号添加到 kernel/syscall.h

```
#define SYS_sysinfo 23
```

在 kernel/kalloc.c 和 kernel/proc.c 中分别添加函数可用的内存数和获取正在使用的进程，并将函数声明添加在 kernel/defs.h

```
kernel/kalloc.h
uint64
free_mem(void)
{
    struct run *r;
    uint64 free = 0;
    acquire(&kmem.lock);
    r = kmem.freelist;
    while (r) {
        free += PGSIZE;
        r = r->next;
    }
    release(&kmem.lock);
    return free;
}

kernel/proc.h
int
count_proc(void)
{
    struct proc *p;
    int count = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
        if (p->state != UNUSED)
            count++;
    }
    return count;
}

kernel/defs.h
// kalloc.c
uint64      free_mem(void);
// proc.c
int         count_proc(void);
```

在 kernel/sysproc.c 文件中添加 sys_sysinfo 函数实现。

```
uint64
sys_sysinfo(void)
{
    struct sysinfo info;
    // 获取系统信息
    info.freemem = free_mem();
}
```

```

info.nproc = count_proc();
uint64 addr;
if (argaddr(0, &addr) < 0)
    return -1;
// 拷贝结构体到用户空间
if (copyout(myproc()->pagetable, addr, (char *)&info, sizeof(info)) < 0)
    return -1;
return 0;
}

```

修改 kernel/syscall.c 文件，在 syscall 表中注册系统调用函数 sys_sysinfo，使得当用户态调用 sysinfo()，内核执行 sys_sysinfo() 函数。添加 sys_sysinfo 函数声明，在 static uint64 (*syscalls[])(void) 和 static char *syscall_names[] 内按照指定格式分别注册。

```

extern uint64 sys_sysinfo(void)
[SYS_sysinfo] sys_sysinfo
[SYS_sysinfo] "sys_sysinfo"

```

在 xv6 的 qemu 环境以下中运行测试命令 sysinfotest，观察到测试成功。

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sysinfotest
sysinfotest: start
sysinfotest: OK
$ 

```

[实验中遇到的问题和解决方法]

我在最开始统计空闲内存的时候，一开始直接遍历 kmem.freelist 计数而没有加锁，结果运行 sysinfotest 的结果不太稳定，有时可以，有时却不能通过。后来才反应过来，kalloc.c 里的空闲链表是多个进程可能访问的共享资源，不加锁的话可能存在并发访问问题。加上 acquire(&kmem.lock) 和 release(&kmem.lock) 之后，测试结果就稳定了。

[实验心得]

这次实验让我对系统调用的理解更具体了。实现系统调用时不只是简单加个函数，还得考虑内核怎么跟用户空间交互另外，我也体会到内核里各模块是相互配合的，要统计内存需要在 kalloc.c 中实现，统计进程需要在 proc.c 中实现，最后在 sysproc.c 里把这些信息整合起来。这种跨文件的协作，让我更清楚 xv6 内

核的结构不是零散的，每个部分都有它的作用。最后我深刻意识到，内核里的共享资源一定要注意保护，像空闲内存链表的锁，少了这一步就会出各种奇怪的问题。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。
然后执行命令 `./grade-lab-syscall`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-syscall
make: 'kernel/kernel' is up to date.
== Test trace 32 grep == trace 32 grep: OK (1.2s)
== Test trace all grep == trace all grep: OK (0.8s)
== Test trace nothing == trace nothing: OK (1.1s)
== Test trace children == trace children: OK (7.1s)
== Test sysinfotest == sysinfotest: OK (0.8s)
== Test time ==
time: OK
Score: 35/35
```

最后在当前分支（`syscall`）下，依次执行以下命令，提交分支到 `github`：

```
$ git checkout syscall #确保在 syscall 分支
$ make clean          #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: system calls."
$ git push github syscall
```

Lab: page tables

实验开始前，依次执行以下命令，切换到 pgtbl 分支

```
$ git fetch
$ git checkout pgtbl
$ make clean
```

Speed up system calls

[实验目的]

Your first task is to implement this optimization for the getpid() system call in xv6.

When each process is created, map one read-only page at USYSCALL (a VA defined in memlayout.h). At the start of this page, store a struct usyscall (also defined in memlayout.h), and initialize it to store the PID of the current process. For this lab, ugetpid() has been provided on the userspace side and will automatically use the USYSCALL mapping. You will receive full credit for this part of the lab if the ugetpid test case passes when running pgtbltest.

您的首要任务是实现此 针对 XV6 中的 getpid () 系统调用进行了优化。

在创建每个进程时，在 USYSCALL（定义于 memlayout.h 的虚拟地址）处映射一个只读页。该页起始位置存储一个 struct usyscall（同样定义于 memlayout.h），并将其初始化为当前进程的 PID。本实验中，用户空间已提供 ugetpid() 函数，该函数将自动使用 USYSCALL 映射。如果运行 pgtbltest 时 ugetpid 测试用例通过，即可获得此部分实验的满分。

[实验内容]

首先修改 kernel/proc.h，给 struct proc 添加共享页字段

```
#ifdef LAB_PGTBL
struct usyscall *usyscall;           // 指向 usyscall 共享页的物理地址
#endif
```

然后修改 kernel/proc.c 中 allocproc() 函数，在 p->pid = allocpid() 之后、p->pagetable = proc_pagetable(p) 之前完成功能片段，为每个进程分配一个只读的共享页，保存进程的 pid，用于用户空间免系统调用读取。

```
#ifdef LAB_PGTBL
// Allocate a usyscall page.
```



```

if((p->usyscall = (struct usyscall *)kalloc()) == 0) {
    freeproc(p);
    release(&p->lock);
    return 0;
}
memset(p->usyscall, 0, PGSIZE);
p->usyscall->pid = p->pid;
#endif

```

修改 kernel/proc.c 中 freeproc(struct proc *p) 函数，在 proc_freepagetable(p->pagetable, p->sz)之前插入功能片段，释放 usyscall 页占用的物理内存，防止内存泄漏。

```

#ifdef LAB_PGTBL
if (p->usyscall)
    kfree(p->usyscall);
p->usyscall = 0;
#endif

```

修改 kernel/proc.c 中 proc_pagetable(struct proc *p)函数，在 trampoline 映射之后，trapframe 映射之前插入功能片段，将物理地址 p->usyscall 映射到用户虚拟地址 USYSCALL，供用户代码直接读取，避免 ecall 系统调用开销。

```

#ifdef LAB_PGTBL
if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)p->usyscall, PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, USYSCALL, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}
#endif

```

修改 kernel/proc.c 中 proc_freepagetable(pagetable_t pagetable, uint64 sz)函数，在释放 trampoline 映射之前插入功能片段，移除 usyscall 页的用户虚拟地址映射，防止页表残留。

```

#ifdef LAB_PGTBL
uvmunmap(pagetable, USYSCALL, 1, 0);
#endif

```

完成上述全部修改后，在 xv6 的 qemu 环境中运行 pgtbltest，观察到 ugetpid 测试运行通过。

```

$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=4

```

完成实验后，还需要回答以下问题：

Which other xv6 system call(s) could be made faster using this shared page?

Explain how.

除了 `getpid()` 之外，其他只涉及读取内核中某些简单信息而无需进行写操作的系统调用，也可以借助共享的只读页面进行加速。其中较为典型的是获取父进程 ID `getppid()`和获取系统运行时间。

`getppid()` 的返回值是当前进程结构中已经存在的父进程 ID 字段，因此可以在 `struct usyscall` 中增加一个 `ppid` 字段，每次调度或 `fork` 时更新即可，从而允许用户程序直接通过共享内存访问该值，避免进入内核态。

同样，`uptime()` 本质是读取 `ticks`(时钟中断次数)，该值在内核中不断更新，可以映射到该共享页的一个只读字段上，由内核定时更新用户空间可见的值，从而实现无系统调用读取当前时间。

此外，像 `getuid()` 等简单身份查询调用也具有类似特性，适合在共享页中以只读方式公开，使系统调用的调用路径从陷入内核转变为纯用户态内存访问，从而显著提升性能。

[实验中遇到的问题和解决方法]

该实验开始时，我完成所有代码编写后，发现 `kernel/proc.c` 添加片段显示存在错误：不允许使用指向不完整类型 "`struct usyscall`" 的指针或引用 C/C++(833)；这种错误一般是 `struct usyscall` 未定义，点开 `memlayout.c` 文件才发现结构体 `usyscall` 已经被定义，只是它条件编译代码，为了和原来的代码保持一致，我将自己添加的代码也改为了条件编译，程序成功执行。

[实验心得]

这次实验通过优化 `getpid()` 系统调用，让我深入理解了提升系统效率的方法。通过在 `USYSCALL` 地址映射包含 `PID` 的只读页，用户程序可直接读取内存获取 `PID`，无需陷入内核，显著减少了系统开销。

此外，我意识到进程生命周期中的资源管理也尤为重要。从分配 `usyscall` 页到解除映射再到释放内存，每个环节都需严格对应，否则会导致内存泄漏或系统不稳定。同时，条件编译的使用则让我学会了如何在不影响原有功能的前提下添加新特性。

总的来说，这次实验加深了我对操作系统内存管理、系统调用优化和资源全生命周期管理的理解，也体会到了内核开发中效率与安全的平衡之道。

Print a page table

[实验目的]

Define a function called `vmprint()`. It should take a `pagetable_t` argument, and print that pagetable in the format described below. Insert `if(p->pid==1)` `vmprint(p->pagetable)` in `exec.c` just before the `return argc`, to print the first process's page table. You receive full credit for this part of the lab if you pass the `pte printout` test of `make grade`.

定义一个名为 `vmprint()` 的函数。该函数接收一个 `pagetable_t` 类型的参数，并按照指定格式打印页表内容。在 `exec.c` 文件中，于 `return argc` 语句前插入：`if(p->pid==1) vmprint(p->pagetable);` 以打印第一个进程的页表。若通过 `make grade` 的页表项（`pte`）打印测试，即可获得此部分实验的满分。

[实验内容]

参考 `kernel/vm.c` 下的 `freewalk()` 函数，了解 `xv6` 的三级页表其实是类似于多叉树的递归结构。`freewalk()` 具体的工作就是递归地去释放每张 2 级或 1 级页表，`xv6` 三级页表的逻辑图如下

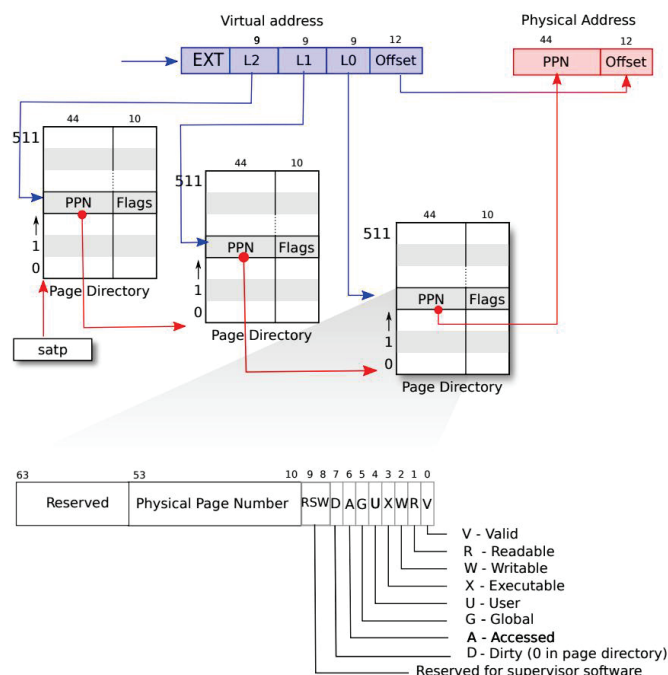


Figure 3.2: RISC-V address translation details.

根据提示，我们首先需要在 `kernel/defs.h` 中声明 `vmprint()` 函数，这样就可以在文件 `exec.c` 中调用。

```
void vmprint(pagetable_t pagetable);
```

在 `kernel/vm.c` 中实现 `vmprint()`

```
static void vmprint_helper(pagetable_t pagetable, int level) {
```

```

if (level > 2) return;
for (int i = 0; i < 512; i++) {
    pte_t pte = pagetable[i];
    if (pte & PTE_V) {
        uint64 pa = PTE2PA(pte);
        // 缩进处理
        for (int j = 0; j <= level; j++) {
            printf(" ..");
        }
        printf("%d: pte %p pa %p\n", i, pte, pa);
        // 如果是页表项，递归下一层
        if ((pte & (PTE_R | PTE_W | PTE_X)) == 0) {
            vmprint_helper((pagetable_t)pa, level + 1);
        }
    }
}
}
}

void vmprint(pagetable_t pagetable) {
    printf("page table %p\n", pagetable);
    vmprint_helper(pagetable, 0);
}

```

这段代码由两个函数组成，分别是 `vmprint` 和 `vmprint_helper`。其中，`vmprint` 是页表打印的入口函数，它接收一个页表指针 `pagetable`，首先输出该页表的地址，然后调用辅助函数 `vmprint_helper` 对整个页表进行递归打印。

`vmprint_helper` 负责遍历页表中的所有 512 项。对每个页表项，如果其有效位 `PTE_V` 被置位，说明该项有效，就打印该项的索引、PTE 值及其对应的物理地址。打印时根据页表的层级深度加上相应缩进，以便清晰展示页表结构。若该页表项没有读、写、执行权限，即不是映射数据页而是指向下一级页表，则递归调用自身处理下一级页表。递归深度最多为三级以符合 RISC-V Sv39 的三级页表结构。

打开 `kernel/exec.c`，在 `exec()` 函数返回(`return argc`; 之前)添加内容，以打印第一个进程的页表。

```

if (p->pid == 1)
    vmprint(p->pagetable);

```

完成上述全部修改后，运行 `make qemu` 指令便会得到以下结果：

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000
.. .. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh

```

完成试验后，还需要回答以下问题：

Explain the output of `vmprint` in terms of Fig 3-4 from the text. What does page 0 contain? What is in page 2? When running in user mode, could the process read/write the memory mapped by page 1? What does the third to last page contain?

`vmprint()` 输出的是进程页表的层级结构，遵循 RISC-V 的三层页表设计。最顶层是页目录（L2），每个条目指向中间页表（L1），而中间页表条目进一步指向页表（L0），L0 中每个条目最终映射一个物理页帧。

在实验示例输出中，“page 0”表示第一级页表（L2）的第 0 项，该项有效，指向第二级页表（L1）。L1 的第 0 项同样有效，指向第三级页表（L0）。L0 的第 0、1、2 项分别映射有效物理页帧，即用户程序实际使用的前 3 页虚拟地址空间。也就是说，page 0 对应的是最上层页表项，它的有效项表示该地址空间中被使用的范围。

“page 2”对应的是 L0 的第二个页表项，即用户程序虚拟地址的第三页。

“page 1”也是 L0 中的页表项，映射的是用户空间的第二页。从页表项中提取的 PTE 权限位可以确定该页在用户态下是否可读写。若该 PTE 中包含 PTE_U 和 PTE_R，说明在用户模式下可读取；若同时包含 PTE_W，则可写。因此是否可读写要具体分析该页的 PTE 权限位。

“第三到最后一个页面”是指页表输出中倒数第三个有效叶子节点。在 xv6 中，倒数的几个映射通常对应 trampoline 或内核栈区域。但若该条目出现在用户页表，而不是内核页表，并且其物理地址落在内核段范围，它可能是共享的 trampoline 映射或用户访问不了的内核地址，通常设为只执行不可读写，用于用户态到内核态切换。通过页表权限位可以确认其不可访问性，验证其特殊用途。

[实验中遇到的问题和解决方法]

这个实验比较简单，使用递归方法可以很好解决。

[实验心得]

这次实验通过实现 `vmprint()` 函数打印页表，让我对 xv6 的三级页表结构有了直观认识。递归遍历页表项的过程中，清晰看到了页表如何层级指向：一级页表项指向二级页表，二级指向三级，最终三级页表项指向数据页，而 `PTE_V` 等标志位则明确区分了有效页表项与无效项、页表页与数据页。在 `exec.c` 中打印第一个进程的页表时，实际输出与理论的三级页表结构对应上了，让抽象的页表概念变得具体。同时，递归函数的设计也帮我更好理解了页表的树形组织，每一级的缩进打印更直观展现了层级关系。

总的来说，这次实验让我从代码层面掌握了页表的遍历方法，加深了对内存地址转换中页表作用的理解，也体会到标志位在页表项中的关键作用。

Detecting which pages have been accessed

[实验目的]

Your job is to implement `pgaccess()`, a system call that reports which pages have been accessed. The system call takes three arguments. First, it takes the starting virtual address of the first user page to check. Second, it takes the number of pages to check. Finally, it takes a user address to a buffer to store the results into a bitmask (a datastructure that uses one bit per page and where the first page corresponds to the least significant bit). You will receive full credit for this part of the lab if the `pgaccess` test case passes when running `pgtbltest`.

你的任务是实现 `pgaccess()`，这是一个报告哪些页面被访问过的系统调用。该系统调用接受三个参数。第一，它接受待检查的第一个用户页的起始虚拟地址。第二，它接受需要检查的页数。最后，它接受一个指向缓冲区的用户地址，用于将结果存储为位掩码（一种每页使用一个比特的数据结构，其中第一页对应最低有效位）。当运行 `pgtbltest` 时，如果 `pgaccess` 测试用例通过，你将获得本实验部分的满分。

[实验内容]

1. 在 `kernel/riscv.h` 中添加访问位 `PTE_A` 的定义（RISC-V 定义中该位为第 6 位，即 $1L \ll 6$

```
#define PTE_A (1L << 6) // accessed
```

2. 在 kernel/sysproc.c 中实现 sys_pgaccess()

```
#ifdef LAB_PGTBL
extern pte_t* walk(pagetable_t pagetable, uint64 va, int alloc);
int
sys_pgaccess(void)
{
    uint64 base;          // 起始虚拟地址
    int len;              // 检查页数
    uint64 user_mask;     // 用户空间掩码地址
    if (argaddr(0, &base) < 0 || argint(1, &len) < 0 || argaddr(2, &user_mask) < 0)
        return -1;
    if (len < 0 || len > 64) // 最多 64 页，适配 64 位掩码
        return -1;
    struct proc *p = myproc();
    uint64 mask = 0;
    for (int i = 0; i < len; i++) {
        uint64 va = base + i * PGSIZE;
        pte_t *pte = walk(p->pagetable, va, 0);
        if (pte && (*pte & PTE_V)) {
            if (*pte & PTE_A) {
                mask |= (1UL << i); // 设置对应位
                *pte &= ~PTE_A;     // 清除访问位
            }
        }
    }
    if (copyout(p->pagetable, user_mask, (char *)&mask, sizeof(mask)) < 0)
        return -1;
    return 0;
}
#endif
```

`sys_pgaccess` 系统调用的代码实现聚焦于页访问状态的检测与反馈，整体逻辑围绕参数校验、页表遍历和访问位操作展开，且通过条件编译（`#ifdef LAB_PGTBL`）确保新功能不影响原有内核结构。首先，函数通过 `argaddr` 和 `argint` 获取用户传入的三个核心参数：起始虚拟地址 `base`、待检测的页数 `len`，以及存储结果的用户空间地址 `user_mask`。参数校验环节尤为严谨：若 `argaddr` 或 `argint` 返回负值（表示参数获取失败），或 `len` 为负数、超过 64（超出位掩码可表示的范围），函数直接返回 -1，避免无效参数进入后续逻辑，体现了内核对用户输入的安全防护。

核心的页访问检测逻辑通过循环实现：对从 `base` 开始的 `len` 个页（每个页大小为 `PGSIZE`），逐个计算虚拟地址 `va = base + i * PGSIZE`，再调用 `walk` 函数从当前进程的页表（`p->pagetable`）中查找对应的页表项 `pte`。`walk` 函数的作用是逐

层解析三级页表，定位到 `va` 对应的 `pte`，其声明通过 `extern` 引入，确保编译时可识别。找到 `pte` 后，先验证其有效性 (`*pte & PTE_V`) —— 只有有效页表项才可能被访问；若有效，再检查访问位 `PTE_A`（此前在 `riscv.h` 中定义为 $1L \ll 6$ ，对应 RISC-V 的硬件置位规则）：若 `PTE_A` 置位，说明该页被访问过，便在 `mask` 的第 `i` 位（对应第 `i` 个页）置 1，同时清除 `PTE_A` (`*pte &= ~PTE_A`)，以便后续检测能准确反映新的访问情况。

最后，函数通过 `copyout` 将 `mask` 从内核空间拷贝到用户指定的 `user_mask` 地址，完成结果传递；若拷贝失败（`copyout` 返回负值），则返回 -1，确保用户能感知到数据传递异常。整体代码既利用了硬件自动置位的 `PTE_A` 实现高效跟踪，又通过软件逻辑完成状态解析与反馈，形成了一套完整的页访问检测机制。

完成上述全部修改后，在 `xv6` 的 `qemu` 环境中运行 `pgtbltest`，观察到 `pgaccess` 测试运行通过。

```
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgaccess_test: OK
pgtbltest: all tests succeeded
```

[实验中遇到的问题和解决方法]

实验中最初遇到的问题是编译时出现“`walk` 函数未声明”的错误。排查发现，虽然代码中调用了 `walk` 函数，但未在当前文件（`sysproc.c`）或者 `defs.h` 中声明其原型，导致编译器无法识别。我在 `sys_pgaccess` 函数前添加 `extern pte_t* walk(pagetable_t pagetable, uint64 va, int alloc);`，显式声明 `walk` 的接口，确保编译通过。

[实验心得]

这次实验让我对页表访问跟踪的实现机制有了更具体的认知。`sys_pgaccess` 的核心是借助硬件自动维护的 `PTE_A` 位，通过软件逻辑完成“检测 - 记录 - 重置”的闭环：硬件在页被访问时自动置位，软件遍历页表检查该位状态，再清除位以便后续检测，这种“软硬协同”的设计既高效又灵活，让内核能低成本地追踪页的使用情况。此外，`walk` 函数的使用让我再次熟悉了页表的层级查找逻辑。从一级页表到二级、三级，最终定位到物理页，这一过程是虚拟地址映射为物理地址的具体体现。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。

然后创建一个 `answers-pgtbl.txt` 文件，在其中存放本实验两个问题的答案 (Speed up system calls、Print a page table 小实验最后的问题)。

然后执行命令 `./grade-lab-pgtbl`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-pgtbl
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (1.2s)
== Test  pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
== Test  pgtbltest: pgaccess ==
pgtbltest: pgaccess: OK
== Test pte printout == pte printout: OK (1.0s)
== Test answers-pgtbl.txt == answers-pgtbl.txt: OK
== Test usertests == (85.3s)
== Test  usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 46/46
```

最后在当前分支 (pgtbl) 下，依次执行以下命令，提交分支到 github:

```
$ git checkout pgtbl  #确保在 pgtbl 分支
$ make clean          #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: page tables."
$ git push github pgtbl:pgtbl
```

Lab: traps

实验开始前，依次执行以下命令，切换到 traps 分支

```
$ git fetch
$ git checkout traps
$ make clean
```

RISC-V assembly

[实验目的]

It will be important to understand a bit of RISC-V assembly, which you were exposed to in 6.004. There is a file `user/call.c` in your xv6 repo. `make fs.img` compiles it and also produces a readable assembly version of the program in `user/call.asm`.

Read the code in `call.asm` for the functions `g`, `f`, and `main`. The instruction manual for RISC-V is on the reference page. You should answer some questions in this part. (store the answers in a file `answers-traps.txt`)

了解一些 RISC-V 汇编是很重要的，你在 6.004 课程中接触过它。你的 xv6 代码仓库中有一个文件 `user/call.c`。运行 `make fs.img` 会编译它，并在 `user/call.asm` 中生成该程序的一个可读的汇编版本。

阅读 `call.asm` 中函数 `g`、`f` 和 `main` 的代码。RISC-V 的指令手册在参考页面上。你应该在本部分回答一些问题。（将答案存储在文件 `answers-traps.txt` 中）

[实验内容]

首先使用 `make fs.img` 指令编译文件 `user/call.c`，生成可读的汇编程序文件 `user/call.asm`。

```
● bruce@Jarvis:~/xv6-labs-2021$ make fs.img
make: 'fs.img' is up to date.
```

在 `user/call.asm` 中可以找到函数 `g`、`f` 和 `main` 的代码

```
int g(int x) {
0:      1141          addi    sp,sp,-16
2:      e422          sd      s0,8(sp)
4:      0800          addi    s0,sp,16
return x+3;
}
6:      250d          addiw   a0,a0,3
```

```

8:      6422          ld      s0,8(sp)
a:      0141          addi     sp,sp,16
c:      8082          ret

000000000000000e <f>:

int f(int x) {
    e:      1141          addi     sp,sp,-16
10:      e422          sd      s0,8(sp)
12:      0800          addi     s0,sp,16
    return g(x);
}
14:      250d          addiw    a0,a0,3
16:      6422          ld      s0,8(sp)
18:      0141          addi     sp,sp,16
1a:      8082          ret

000000000000001c <main>:

void main(void) {
    1c:      1141          addi     sp,sp,-16
1e:      e406          sd      ra,8(sp)
20:      e022          sd      s0,0(sp)
22:      0800          addi     s0,sp,16
    printf("%d %d\n", f(8)+1, 13);
24:      4635          li      a2,13
26:      45b1          li      a1,12
28:      00000517        auipc    a0,0x0
2c:      7b050513        addi     a0,a0,1968 # 7d8 <malloc+0xea>
30:      00000097        auipc    ra,0x0
34:      600080e7        jalr     1536(ra) # 630 <printf>
    exit(0);
38:      4501          li      a0,0
3a:      00000097        auipc    ra,0x0
3e:      27e080e7        jalr     638(ra) # 2b8 <exit>

```

阅读代码后，回答以下问题

1. Which registers contain arguments to functions? For example, which register holds 13 in main's call to printf?

RISC-V 的调用约定规定函数参数通过寄存器 a0 到 a7（即 x10 到 x17）传递。

在 main 中调用 printf("%d %d\n", f(8)+1, 13)时，参数依次放入 a0、a1、a2。由汇编代码 24: 4635 li a2,13 可知，立即数 13 则放入 a2 寄存器。

2. Where is the call to function `f` in the assembly code for `main`? Where is the call to `g`? (Hint: the compiler may inline functions.)

在汇编文件 `call.asm` 中, `main` 函数没有对 `f` 或 `g` 的显式调用指令 (即 `jal f` 或 `jal g`)。这是因为 `f` 和 `g` 都非常简单, 编译器将它们内联到了调用处, 直接计算了结果 $f(8)+1=12$, 然后将其赋值给寄存器。因此它们的调用被优化掉, 不会出现在汇编中。

3. At what address is the function `printf` located?

以下这段代码为跳转到 `printf` 对应位置:

```
34:      600080e7      jalr      1536(ra) # 630 <printf>
```

由汇编代码可知 `printf` 函数的地址为 `ra+1536`, 查看上一条汇编代码

```
30:      00000097      auipc    ra, 0x0
```

可知此时 `ra` 的值即为 `0x30`, 因此 `printf` 函数的地址为 $0x30+1536=0x630$ 。

4. What value is in the register `ra` just after the `jalr` to `printf` in `main`?

`jalr` 指令的作用是把下一条指令的地址 (即 $PC+4$) 保存在返回地址寄存器 `ra` 中并跳转到指定地址。根据 `jalr` 指令的功能, 在刚跳转后 `ra` 的值为 $pc+4=0x34+4=0x38$ 。

5. Run the following code.

```
unsigned int i = 0x00646c72;
printf("H%x Wo%s", 57616, &i);
```

What is the output? The output depends on that fact that the RISC-V is little-endian. If the RISC-V were instead big-endian what would you set to in order to yield the same output? Would you need to change to a different value?

`%x` 表示以十六进制格式输出整数 `57616`, 即输出 `e110`; `%s` 从地址 `&i` 开始, 将连续内存解释为字符串 (以 `\0` 结尾的字节序列)。 `i = 0x00646c72`, 对应 16 字节进制 `00 64 6c 72`, 按小端存储顺序, 其在内存中按顺序为 `72 6c 64 00`, 分别对应 `'r'`、`'l'`、`'d'`、`\0`, 因此 `%s` 输出 `"rld"`。故最终输出为 `"He110 World"`。

如果是大端系统, 影响 `%s` 的输出, 第一个存储的字节为 `\0`, 因此不会再输出其它字节, 输出结果为 `"He110 Wo"`。

为了得到和小端一样的 `"rld"` 输出, 需要让内存中按顺序存储 `72 6c 64 00`, 即令 `unsigned int i = 0x726c6400`。无需修改除 `i` 外的值。

6. In the following code, what is going to be printed after `'y'=?` (note: the answer is not a specific value.) Why does this happen?

```
printf("x=%d y=%d", 3);
```

y 输出的是一个受调用前代码影响的“随机”值，因为 `printf` 根据格式字符串尝试读取两个整数参数，但程序只传入了一个。根据 RISC-V ABI 规定，两个整数参数分别通过 `a1` 和 `a2` 传递，第一个参数 3 被正确传入 `a1`，但 `a2` 没有被设置，因此 `printf` 会从 `a2` 中取出一个未定义的值，用于第二个 `%d`，结果是随机的、不可靠的。

[实验中遇到的问题和解决方法]

本次实验主要是汇编代码的阅读和理解，没有遇到什么问题。

[实验心得]

通过这次实验，我对 RISC-V 汇编的理论逻辑有了更清晰的认识。作为精简指令集架构，其函数参数通过 `a0` 至 `a7` 寄存器传递，这一设计源于寄存器访问速度远快于内存，这样做能减少数据交互开销，提升执行效率。在函数调用中，`jalr` 指令将返回地址存入 `ra` 寄存器，配合栈帧管理实现嵌套执行，既保证流程连贯，又形成统一规范，体现了底层设计的严谨性。

这些规则皆基于硬件特性与效率需求，让我更理解了指令集架构在软硬件交互中的核心作用。

Backtrace

[实验目的]

For debugging it is often useful to have a backtrace: a list of the function calls on the stack above the point at which the error occurred. Implement a `backtrace()` function in `kernel/printf.c`. Insert a call to this function in `sys_sleep`, and then run `bttest`, which calls `sys_sleep`. The compiler puts in each stack frame a frame pointer that holds the address of the caller's frame pointer. Your backtrace should use these frame pointers to walk up the stack and print the saved return address in each stack frame.

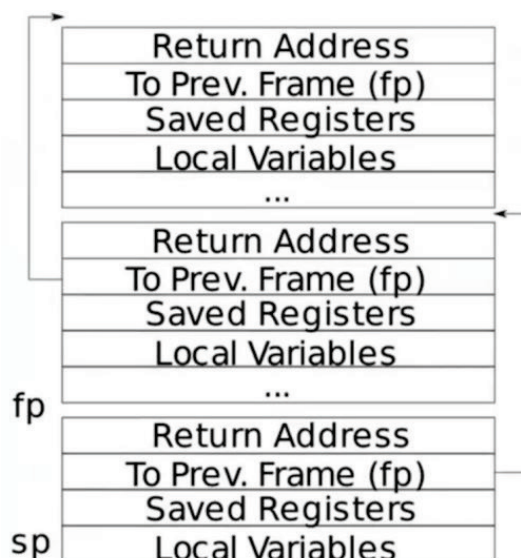
对于调试，拥有一个回溯通常是很有用的：错误发生点之上的栈上函数调用的列表。在 `kernel/printf.c` 中实现一个 `backtrace()` 函数。在 `sys_sleep` 中插入一个对这个函数的调用，然后运行 `bttest`，它调用了 `sys_sleep`。编译器在每个栈帧中放入一个帧指针，该指针持有调用者的帧指针的地址。你的回溯应该使用这些帧指针向上遍历栈并在每个栈帧中打印保存的返回地址。

[实验内容]

在编写代码前，首先需要对栈帧结构和函数调用机制有基本了解。在 RISC-V 架构中，栈帧是函数调用时在栈上分配的内存区域，用于保存函数执行的上下文信息，其结构遵循固定规范：每个栈帧会保存上一级函数的帧指针（即调用者的 `s0` 寄存器值）、当前函数的返回地址（即调用者的 `ra` 寄存器值），以及局部变量等数据。

当发生函数调用时，调用方会将返回地址（`ra`）压入栈中，再将自身的帧指针（`s0`）压入栈，随后为被调用函数分配新的栈帧并更新 `s0` 指向新栈帧。

关于栈帧内的地址偏移，返回地址（`ra`）通常存储在当前帧指针下方 8 字节处（`fp-8`），这是因为 RISC-V 的栈按 8 字节对齐，且返回地址在函数调用时优先入栈；上一级帧指针（调用者的 `s0`）则存储在 `fp-16` 处，紧随返回地址之后入栈。



在 `kernel/riscv.h` 末尾添加函数 `r_fp()`，用于获取当前函数的帧指针(`s0`)

```
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x));
    return x;
}
```

在 `kernel/printf.c` 中实现函数 `backtrace()` 函数的作用是遍历并打印当前程序的调用栈，帮助调试程序。

```
void
backtrace()
{
```

```

printf("backtrace:\n");
uint64 fp = r_fp(); // 当前帧指针
uint64 stack_top = PGROUNDUP(fp); // 当前栈页的顶部地址
uint64 stack_bottom = PROUNDDOWN(fp); // 当前栈页的底部地址
while (fp >= stack_bottom + 16 && fp < stack_top) {
    uint64 ra = *(uint64*)(fp - 8); // 取出当前栈帧中的返回地址
    printf("%p\n", ra); // 打印返回地址
    fp = *(uint64*)(fp - 16); // 取出上一层帧指针，进入上层栈帧
}
}

```

在 kernel/defs.h 中添加 backtrace() 函数声明

```
void backtrace(void);
```

在 kernel/sysproc.c 的 sys_sleep() 中添加对 backtrace() 调用，修改后的 sys_sleep() 函数为

```

uint64
sys_sleep(void)
{
    int n;
    uint ticks0;
    backtrace();
    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}

```

在 xv6 的 qemu 环境以下中输入 bttest 命令，观察到输出三个地址，然后退出 qemu，然后执行 addr2line -e kernel/kernel 命令，依次输入上述三个地址，观察到通过测试。

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ bttest
backtrace:
0x00000000800020cc
0x0000000080001fa6
0x0000000080001c90
$ QEMU: Terminated
● bruce@Jarvis:~/xv6-labs-2021$ addr2line -e kernel/kernel
0x00000000800020cc
/home/bruce/xv6-labs-2021/kernel/sysproc.c:62
0x0000000080001fa6
/home/bruce/xv6-labs-2021/kernel/syscall.c:140
0x0000000080001c90
/home/bruce/xv6-labs-2021/kernel/trap.c:76
○ bruce@Jarvis:~/xv6-labs-2021$

```

[实验中遇到的问题和解决方法]

本次实验的重点在于理解函数调用栈和栈帧结构的原理，尤其是 `*(uint64*)(fp - 8)` 和 `*(uint64*)(fp - 16)` 分别代表了什么含义。`*(uint64*)(fp - 8)` 获取的是当前栈帧的返回地址，`*(uint64*)(fp - 16)` 用于获取上一个栈帧的帧指针。明白这些后，本次实验便没有什么困难。

[实验心得]

这次实验让我对栈帧结构和函数调用机制有了更深刻的理解。通过手动实现回溯功能，我直观地认识到帧指针作为“调用链锚点”的关键作用：每个栈帧通过保存上一级帧指针，形成了从当前函数到 `main` 的完整调用路径，这一设计使调试时能逆向追踪程序执行流程成为可能。在处理帧指针合法性检查时，我体会到内核编程中内存安全的重要性。一个简单的边界判断错误可能导致内核崩溃，这要求我们要对系统内存布局有清晰认知。

总的来说，这次实验不仅提升了我的编程能力，更让我对计算机系统的运行机制有了更深入的认识，为后续学习打下了坚实基础。

Alarm

[实验目的]

In this exercise you'll add a feature to xv6 that periodically alerts a process as it uses CPU time. This might be useful for compute-bound processes that want to limit how much CPU time they chew up, or for processes that want to compute but also want

to take some periodic action. More generally, you'll be implementing a primitive form of user-level interrupt/fault handlers; you could use something similar to handle page faults in the application, for example. Your solution is correct if it passes `alarmtest` and `usertests`.

You should add a new `sigalarm(interval, handler)` system call. If an application calls `sigalarm(n, fn)`, then after every `n` "ticks" of CPU time that the program consumes, the kernel should cause application function `fn` to be called. When `fn` returns, the application should resume where it left off. A tick is a fairly arbitrary unit of time in `xv6`, determined by how often a hardware timer generates interrupts. If an application calls `sigalarm(0, 0)`, the kernel should stop generating periodic alarm calls.

在该练习中，您将为 `xv6` 添加一项周期性提醒进程使用 CPU 时间的功能。这对于希望限制自身 CPU 时间消耗的计算密集型进程，或需要执行计算但需周期性执行其他操作的进程非常有用。更广泛地说，您将实现一种用户级中断/故障处理程序的初级形式；例如，可利用类似机制处理应用程序中的页错误。若实现能通过 `alarmtest` 和 `usertests` 测试，则视为正确。

您需新增一个 `sigalarm(interval, handler)` 系统调用。当应用程序调用 `sigalarm(n, fn)` 后，程序每消耗 `n` 个 CPU 时间滴答(tick)，内核应触发应用程序函数 `fn` 的调用。`fn` 返回后，应用程序应从中断处继续执行。在 `xv6` 中，滴答是由硬件定时器中断频率决定的相对任意的时间单位。若应用程序调用 `sigalarm(0, 0)`，内核应停止生成周期性警报调用。

[实验内容]

添加系统调用声明，在 `kernel/syscall.h` 中分配新系统调用号，并在在 `kernel/syscall.c` 的 `syscalls[]` 表中添加对应函数：

```
// kernel/syscall.h
#define SYS_sigalarm 22
#define SYS_sigreturn 23

// kernel/syscall.c
extern uint64 sys_sigalarm(void);
extern uint64 sys_sigreturn(void);

// kernel/syscall.c static uint64 (*syscalls[])(void)
[SYS_sigalarm] sys_sigalarm,
[SYS_sigreturn] sys_sigreturn,
```

在 `kernel/sysproc.c` 中实现系统调用接口函数 `sys_sigalarm` 和 `sys_sigreturn`

```
uint64 sys_sigalarm(void) {
```

```

int interval;
uint64 handler;

if (argint(0, &interval) < 0)
    return -1;
if (argaddr(1, &handler) < 0)
    return -1;

struct proc *p = myproc();
p->alarm_interval = interval;
p->alarm_handler = handler;
p->ticks_passed = 0;
p->alarm_active = 0;
return 0;
}

uint64 sys_sigreturn(void) {
    struct proc *p = myproc();
    memmove(p->trapframe, &p->alarm_tf, sizeof(struct trapframe));
    p->alarm_active = 0;
    p->ticks_passed = 0; // 重新计时
    return 0;
}

```

系统调用 `sys_sigalarm` 的实现是用户程序与内核之间注册定时中断回调函数的接口。当用户调用 `sigalarm(n, handler)` 时，该系统调用会提取用户传入的两个参数：一个整数 `n` 表示间隔的时钟 tick 数，另一个是一个函数指针 `handler`，指向用户空间的处理函数。内核会将这两个参数分别保存到当前进程的 `alarm_interval` 和 `alarm_handler` 字段中，并初始化一个 tick 计数器 `ticks_passed` 为 0，用于追踪已过去的时钟中断次数。与此同时，还会将一个名为 `alarm_active` 的标志位设置为 0，以表明当前还未进入 `alarm` 处理状态。。

`sys_sigreturn` 的实现会将之前在 `usertrap()` 中保存的寄存器上下文完整地拷贝回当前 `trapframe` 中，从而恢复中断前的程序状态。这使得程序可以无缝地从中断前的位置继续执行，就好像什么都没有发生过一样。此外，`sigreturn` 还会将 `alarm_active` 标志重置为 0，表示 `handler` 已经完成执行，并将 tick 计数器 `ticks_passed` 清零，为下一次 `alarm` 做好准备。

在 `kernel/proc.h` 的 `struct proc` 中添加 `alarm` 相关字段

```

int alarm_interval;
uint64 alarm_handler;

int ticks_passed;
int alarm_active;

struct trapframe alarm_tf;

```

在 user/user.h 中添加两个系统调用的函数原型

```
int sigalarm(int ticks, void (*handler)());  
int sigreturn(void);
```

然后在 user/usys.pl 脚本中添加两个系统调用的相应 entry

```
entry("sigalarm")  
entry("sigreturn")
```

在进程初始化时，为该指针进行赋值并在进程结束后，释放该指针。即在 kernel/proc.c 中 allocproc(void) 函数和 freeproc(struct proc *p) 分别添加

```
p->alarm_interval = 0;  
p->ticks_passed = 0;  
p->alarm_handler = 0;  
p->alarm_active = 0;
```

在 kernel/trap.c 的函数 usertrap(void) 中实现处理时钟中断逻辑

```
if (which_dev == 2)  
{  
    struct proc *p = myproc();  
    if (p && p->alarm_interval > 0)  
    {  
        p->ticks_passed++;  
        if (!p->alarm_active && p->ticks_passed == p->alarm_interval)  
        {  
            p->alarm_active = 1;  
            p->ticks_passed = 0;  
            memmove(&p->alarm_tf, p->trapframe, sizeof(struct trapframe));  
            p->trapframe->epc = p->alarm_handler;  
        }  
    }  
    yield();  
}
```

usertrap() 函数负责响应所有用户态发起的中断和异常，其中包括定时器中断。当中断来源为时钟设备（which_dev == 2）时，内核将首先判断当前进程是否启用了 alarm 功能（即 alarm_interval 是否大于 0）。如果启用了，内核会自增 ticks_passed 计数器，并判断是否已达到用户设定的触发阈值。如果达到了，并且当前未处于 alarm handler 执行状态（即 alarm_active == 0），则将用户当前的 trapframe 内容完整备份到一个新结构中 alarm_tf，以便稍后恢复。随后将 trapframe 中的 epc 字段重写为用户设定的 handler 地址，从而使得 trap 返回后，用户程序并非从中断前的指令继续，而是自动跳转进入 handler 执行。

修改 Makefile 找到 UPROGS 列表，添加一行 \$U/_alarmtest\

在 xv6 的 qemu 环境以下中分别输入 alarmtest 和 usertests 命令，观察到所有测试都通过了。

```

$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
test1 passed
test2 start
.....alarm!
test2 passed

test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED

```

[实验中遇到的问题和解决方法]

另一个关键问题出现在 `usertrap()` 函数中实现时钟中断触发 `alarm` 的逻辑。当我初次实现时，每当时钟中断到达，我都直接设置 `trapframe->epc = alarm_handler`，导致 `alarm handler` 在返回前程序无法正确恢复——因为我并未保存原 `trapframe` 的寄存器内容。为了解决这个问题，我新建了一个 `alarm_tf` 成员变量用于保存原始 `trapframe`，在触发 `handler` 前用 `memmove()` 拷贝了当前 `trapframe` 的内容，保证 `handler` 执行完毕后可以借助 `sigreturn()` 完整还原执行上下文。这样，用户程序可以从被中断的位置无缝继续执行。

[实验心得]

本次 `Alarm` 实验让我对操作系统中断处理与上下文管理有了深刻认识。通过实现 `sigalarm` 和 `sigreturn` 系统调用，我清晰看到用户态与内核态交互的桥梁：用户程序通过系统调用注册定时参数，内核则在进程控制块中维护 `alarm` 相关状态，体现了进程管理的精细化。整个过程展现了硬件中断、内核响应与用户回调的协同，让我直观理解了操作系统如何在底层机制支撑下，为上层应用提供灵活的定时功能。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。
然后执行命令 `./grade-lab-traps`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-traps
make: 'kernel/kernel' is up to date.
== Test answers-traps.txt == answers-traps.txt: OK
== Test backtrace test == backtrace test: OK (1.0s)
== Test running alarmtest == (3.2s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test usertests == usertests: OK (81.5s)
== Test time ==
time: OK
Score: 85/85
```

最后在当前分支（traps）下，依次执行以下命令，提交分支到 github:

```
$ git checkout traps #确保在 traps 分支
$ make clean #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab:traps."
$ git push github traps:traps
```

Lab: Copy-on-Write Fork for xv6

实验开始前，依次执行以下命令，切换到 cow 分支

```
$ git fetch
$ git checkout cow
$ make clean
```

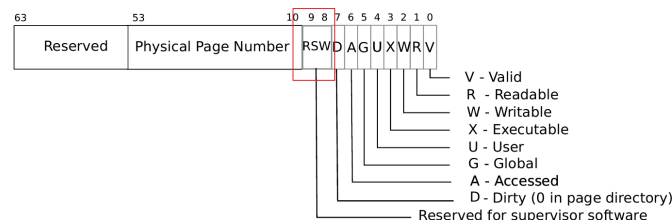
Implement copy-on write

[实验目的]

Your task is to implement copy-on-write fork in the xv6 kernel. You are done if your modified kernel executes both the cowtest and usertests programs successfully.

您的任务是在内核中实现写时复制（copy-on-write）的 fork 功能。当您的修改版内核能够成功执行 cowtest 和 usertests 这两个测试程序时，即表明您已完成修改。

[实验内容]



首先需要为 PTE 添加一个用于识别 COW 的标志位，由于据 PTE 的后 10 位中第 8、9 位是保留没有被使用，因此我们可以用第 8 位来作为 COW 的标志位。

在 kernel/riscv.h 中添加

```
#define PTE_COW (1L << 8)
```

然后增加页面引用计数数组，修改 kernel/kalloc.c 文件，定义引用数组并提供获取索引、增加页面引用计数、减少页面引用计数、获取页面引用计数等操作接口，并在 kernel/defs.h 中声明这些接口。由于该数组为共享资源，还需要为该数组添加锁机制，保证进程互斥性访问该资源。

```
//kernel/kalloc.c
struct {
    struct spinlock lock;
    int refcount[PHYSTOP / PGSIZE]; // 存放每个页面被引用的次数
```

```

} page_ref;
// 获取页面索引
static int
page_index(void *pa)
{
    return ((uint64)pa - (uint64)end) / PGSIZE;
}
void
setref(void *pa)
{
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("ref_init");
    acquire(&page_ref.lock);
    page_ref.refcount[page_index(pa)] = 1;
    release(&page_ref.lock);
}
// 增加引用计数并返回新值
int
incref(void *pa)
{
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("incref");
    acquire(&page_ref.lock);
    int ref = ++page_ref.refcount[page_index(pa)];
    release(&page_ref.lock);
    return ref;
}
// 减少引用计数并返回新值
int
decref(void *pa)
{
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("decref");
    acquire(&page_ref.lock);
    int idx = page_index(pa);
    if(page_ref.refcount[idx] <= 0)
        panic("decref: refcount already zero");
    int ref = --page_ref.refcount[idx];
    release(&page_ref.lock);
    return ref;
}
// 获取引用计数
int

```

```

getref(void *pa)
{
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("getref");
    acquire(&page_ref.lock);
    int ref = page_ref.refcount[page_index(pa)];
    release(&page_ref.lock);
    return ref;
}

//kernel/defs.h
Int      incref(void*);
int      decref(void*);
int      getref(void*);

```

修改 kinit() 和 freerange() 函数，增加初始化引用计数数组逻辑。

```

void
kinit()
{
    initlock(&kmem.lock, "kmem");
    initlock(&page_ref.lock, "page_ref");
    memset(page_ref.refcount, 0, sizeof(page_ref.refcount));
    freerange(end, (void*)PHYSTOP);
}

void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE) {
        setref((void *) p);
        kfree(p);
    }
}

```

修改 kalloc() 函数和 kfree() 函数，增加引用计数实现相关逻辑。kalloc() 将页面计数变为 1，kfree() 时减少页面引用计数，并在计数为 0 时，释放页面。

```

void
kfree(void *pa)
{
    struct run *r;
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");
    // 原子地减少引用计数并检查
    if(decref(pa) > 0) {

```



```

    // 还有其他引用，不释放
    return;
}

// Fill with junk to catch dangling refs.
memset(pa, 1, PGSIZE);
r = (struct run*)pa;
acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
release(&kmem.lock);
}

void *
kalloc(void)
{
    struct run *r;
    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r) {
        kmem.freelist = r->next;
        setref((void*)r);
    }
    release(&kmem.lock);
    if(r)
        memset((char*)r, 5, PGSIZE); // fill with junk
    return (void*)r;
}

```

修改 kernel/vm.c 中的 `uvmcopy()` 函数，当创建子进程时，不再为子进程分配新的物理页面，而是子进程直接映射到父进程相同的物理页面，同时，将可写页面改为只读页面，并标明该页为 COW 页，同时页面计数加一。

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if(flags & PTE_W) {

```

```

    flags = (flags & ~PTE_W) | PTE_COW;
    *pte = PA2PTE(pa) | flags;
}
if(mappages(new, i, PGSIZE, pa, flags) != 0){
    goto err;
}
// 增加页面引用计数
incref((void*)pa);
}
return 0;
err:
uvmunmap(new, 0, i / PGSIZE, 1);
return -1;
}

```

在 kernel/vm.c 中实现页错误处理机制，为进程分配新物理页、复制内容、更新页表，确保进程写的是自己独立的副本而非 COW 项，由函数 cowfault() 实现，同时需要在 声明该函数。

```

//kernel/vm.c
int
cowfault(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    uint64 pa;
    uint flags;
    char *newpa;
    // 地址必须在用户空间范围内
    if(va >= MAXVA)
        return -1;
    // 获取 PTE
    pte = walk(pagetable, va, 0);
    if(pte == 0)
        return -1;
    // 检查页面是否存在且标记为 COW
    if((*pte & PTE_V) == 0 || (*pte & PTE_COW) == 0)
        return -1;
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    // 检查引用计数
    int ref = getref((void*)pa);
    if(ref == 1) {
        // 只有一个引用，直接修改权限即可
        flags = (flags & ~PTE_COW) | PTE_W;
        *pte = PA2PTE(pa) | flags;
        return 0;
    }
}

```

```

}
// 多个引用，需要复制页面
newpa = kalloc();
if(newpa == 0)
    return -1;
// 复制页面内容
memmove(newpa, (char*)pa, PGSIZE);
// 更新 PTE
flags = (flags & ~PTE_COW) | PTE_W;
*pte = PA2PTE(newpa) | flags;
// 减少原页面的引用计数
kfree((void*)pa);
return 0;
}
//kernel/defs.h
int                cowfault(pagetable_t, uint64);

```

`copyout()` 工作在内核态，无法触发 `usertrap`，需要为其单独添加到页错误处理机制的逻辑。

```

int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 n, va0, pa0;
    pte_t *pte;
    while(len > 0){
        va0 = PGROUNDDOWN(dstva);
        pa0 = walkaddr(pagetable, va0);
        if(pa0 == 0)
            return -1;
        // 先检查是否需要处理 COW
        pte = walk(pagetable, va0, 0);
        if(pte && (*pte & PTE_COW)) {
            if(cowfault(pagetable, va0) < 0)
                return -1;
            pa0 = walkaddr(pagetable, va0);
        }
        n = PGSIZE - (dstva - va0);
        if(n > len)
            n = len;
        memmove((void *) (pa0 + (dstva - va0)), src, n);
        len -= n;
        src += n;
        dstva = va0 + PGSIZE;
    }
    return 0;
}

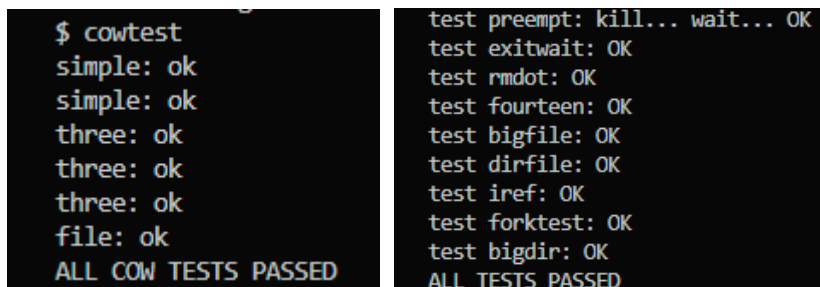
```

```
}
```

最后修改 kernel/trap.c 中 usertrap()函数，在 else if((which_dev = devintr()) != 0)条件处理后加入以下条件处理：

```
else if(r_scause() == 15) {
    uint64 va = r_stval();
    if(cowfault(p->pagetable, va) < 0) {
        printf("usertrap(): unexpected page fault va=%p pid=%d\n", va, p->pid);
        printf("      sepc=%p stval=%p\n", r_sepc(), r_stval());
        p->killed = 1;
    }
}
```

在 xv6 的 qemu 环境以下中分别输入 cowtest 和 usertests 命令，观察到所有测试都通过了。



```
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED

test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

[实验中遇到的问题和解决方法]

此次实验的难度大，代码量也大，需要我们对 Copy-on-Write 有着良好的理解。我在学习了 Copy-on-Write 相关理论知识后，根据教程提示完成了该部分，没有出现知识上的难题。

但初次编写时，由于自己的疏忽，没有看到 kinit() 会间接调用 kfree()，因此开始时将所有页面引用次数设置为 0，导致在初始化后页面引用次数变为 1，与预期不符，总是无法通过测试。后来排查时发现了这一问题并及时纠正，成功通过测试。经过该修改，我的代码成功通过了 cowtest 和 usertests 测试，但是在 ./grade-lab-cow 测试时存在严重的内存泄露问题，经过检查代码，我找到了两个问题：首先，我对于修改后获取引用计数不具有原子性，开始时，我首先通过一个函数修改引用计数，然后再通过一个函数读取引用计数，在多线程任务中，修改后的和读取的可能存在不一致问题，因此，我修改了引用计数修改函数，使其可以直接返回修改后的数值；其次我忽视了对于当 ROW 页重新只有一个进程拥有时，应当恢复该页为普通页、允许写，因此，实际对于这种情况也是重新分配页，成为内存泄露最主要原因。

[实验心得]

本次 Copy-on-Write Fork 实验让我对操作系统中内存管理的优化机制有了深刻理解。写时复制（COW）通过延迟页面复制的时机，不再在 fork 时立即复制所有物理页面，而是让父子进程共享页面并标记为只读 + COW，仅当任一进程写入时才真正复制，大幅降低了 fork 操作的开销。

同时，实验也让我重视细节的重要性，包括引用计数的原子性维护、页表权限的正确修改、异常情况的判断，任何一处疏忽都可能导致系统崩溃或内存泄露。这些经历加深了我对内存管理、进程调度的理解。

提交

首先创建一个 time.txt 文件，在其中存放一个整数，记录实验的小时数。
然后执行命令 `./grade-lab-cow`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-cow
make: 'kernel/kernel' is up to date.
== Test running cowtest == (2.8s)
== Test  simple ==
simple: OK
== Test  three ==
three: OK
== Test  file ==
file: OK
== Test usertests == (88.7s)
(Old xv6.out.usertests failure log removed)
== Test  usertests: copyin ==
usertests: copyin: OK
== Test  usertests: copyout ==
usertests: copyout: OK
== Test  usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 110/110
```

最后在当前分支（cow）下，依次执行以下命令，提交分支到 github:

```
$ git checkout cow    #确保在 cow 分支
$ make clean          #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: Copy-on-Write Fork for xv6."
$ git push github cow:cow
```

Lab: Multithreading

实验开始前，依次执行以下命令，切换到 thread 分支

```
$ git fetch
$ git checkout thread
$ make clean
```

Uthread: switching between threads

[实验目的]

Your job is to come up with a plan to create threads and save/restore registers to switch between threads, and implement that plan. When you're done, make grade should say that your solution passes the uthread test.

你的任务是制定一个线程创建方案，并实现寄存器保存/恢复机制以实现线程切换。完成后，执行 `make grade` 命令应显示你的方案通过了 uthread 测试。

[实验内容]

首先在 `user/uthread.c` 中定义寄存器上下文结构体，用于保存寄存器内容，确保每个线程都有自己的上下文，便于在切换线程时保存和恢复。

```
// 添加寄存器上下文结构体
struct context {
    uint64 ra;
    uint64 sp;
    // callee-saved
    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
};
```

然后线程结构体 `struct thread` 中添加线程上下文字段 `context`，确保每一个线程都有一个上下文结构体。

```
struct thread {
    char    stack[STACK_SIZE]; /* the thread's stack */
    int     state;             /* FREE, RUNNING, RUNNABLE */
    struct context context;
};
```

接着修改 `thread_create()` 函数，确保正确初始化线程上下文。主要时首先将所有寄存器内容归零，然后令 `ra` 和 `sp` 寄存器分别保存 `func` 和栈指针。添加以下代码：

```
memset(&t->context, 0, sizeof(t->context));
t->context.ra = (uint64)func;    // 返回地址设置为线程函数
t->context.sp = (uint64)t->stack + STACK_SIZE; // 栈指针指向栈顶
```

接着修改修改 `thread_schedule()`函数。`thread_schedule()` 负责用户多线程间的调度，在找到线程后就需要进行线程的切换，即需要调用函数 `thread_switch()`。添加以下代码：

```
thread_switch((uint64)&t->context, (uint64)&current_thread->context);
```

最后在 `user/uthread_switch.S` 中实现 `thread_switch` 的汇编代码。

```
.text
/*
 * save the old thread's registers,
 * restore the new thread's registers.
 */
.globl thread_switch
thread_switch:
    /* YOUR CODE HERE */
    /* 保存旧线程的寄存器到 a0 指向的 context */
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)
    /* 从 a1 指向的 context 恢复新线程的寄存器 */
```

```

ld ra, 0(a1)
ld sp, 8(a1)
ld s0, 16(a1)
ld s1, 24(a1)
ld s2, 32(a1)
ld s3, 40(a1)
ld s4, 48(a1)
ld s5, 56(a1)
ld s6, 64(a1)
ld s7, 72(a1)
ld s8, 80(a1)
ld s9, 88(a1)
ld s10, 96(a1)
ld s11, 104(a1)
ret    /* return to ra */

```

在 xv6 的 qemu 环境以下中运行 uthread 命令，得到预期结果，表示测试成功。

```

thread_c 96
thread_a 96
thread_b 96
thread_c 97
thread_a 97
thread_b 97
thread_c 98
thread_a 98
thread_b 98
thread_c 99
thread_a 99
thread_b 99
thread_c: exit after 100
thread_a: exit after 100
thread_b: exit after 100
thread schedule: no runnable threads

```

[实验中遇到的问题和解决方法]

本次实验理解多线程编程，实现用户级线程的上线文切换机制。主要难点在于实现汇编代码的 thread_switch 函数，但是该函数实际上功能与 kernel/swtch.S 中的 swtch 函数一致，且此处 struct context 与内核的 struct context 结构体的成员也相同，因此该函数可以直接复用 kernel/swtch.S 中的 swtch 代码。总体来说并不是很难。

[实验心得]

在进行这个实验的过程中，我对用户级线程系统的设计与实现有了深入的理

解，尤其是线程上下文切换的核心机制。

用户级线程的切换虽与内核级进程切换相通，通过保存当前执行状态、恢复目标执行状态来实现执行流的切换。线程切换的关键在于对上下文的精准管理，每一个线程的上下文都包含了其执行所需的关键信息，确保切换后能无缝继续执行。线程调度器在选择下一个可运行线程后，通过切换函数完成执行权的移交。这个过程中，保存旧线程的寄存器状态与恢复新线程的寄存器状态是核心步骤，任何一个细节的疏漏都可能导致线程执行错乱。

总的来说，这个实验让我清晰认识到，用户级线程通过在用户态实现上下文切换，避免了内核态切换的开销，同时也遵循着与内核进程调度相似的设计原则。线程作为 CPU 执行流的抽象，其切换机制是多线程编程的核心，理解这一机制对于掌握并发程序设计至关重要。

Using threads

[实验目的]

In this assignment you will explore parallel programming with threads and locks using a hash table. You should do this assignment on a real Linux or MacOS computer (not xv6, not qemu) that has multiple cores. Most recent laptops have multicore processors.

在本次作业中，你将通过哈希表探索使用线程和锁的并行编程。你应当在一台具有多个核心的真实 Linux 或 MacOS 计算机上完成本次作业（不是 xv6，也不是 qemu）。大多数最新的笔记本电脑都配备了多核处理器。

[实验内容]

使用如下命令构建程序，该程序包含一个线程不安全的哈希表。

```
$ make ph
```

运行 `./ph 1` 命令，即使用单线程运行该哈希表。输出如下

```
● bruce@Jarvis:~/xv6-labs-2021$ ./ph 1
100000 puts, 4.072 seconds, 24557 puts/second
0: 0 keys missing
100000 gets, 4.325 seconds, 23121 gets/second
```

运行 `./ph 1` 命令，即使用两个线程运行该哈希表，输出如下，可以看到其 put 速度近乎先前 2 倍，但是有 16662 个键丢失，也说明了该哈希表非线程安全。

```

● bruce@Jarvis:~/xv6-labs-2021$ ./ph 2
100000 puts, 1.699 seconds, 58864 puts/second
0: 16662 keys missing
1: 16662 keys missing
200000 gets, 4.205 seconds, 47565 gets/second

```

接下来需要使用互斥锁 `pthread_mutex_t` 来进行数据一致性的保护根据实验指导的提示。在 `notxv6/ph.c` 中构建了一个互斥锁数组。每个 `bucket` 对应一个互斥锁。

```
pthread_mutex_t locks[NBUCKET];
```

然后，需要在 `main()` 函数中对所有互斥锁进行初始化。在创建线程之前添加如下代码：

```

for(int i = 0; i < NBUCKET; i++) {
    pthread_mutex_init(&locks[i], NULL);
}

```

线程的安全问题是对桶中的链表进行操作而导致的，因此需要修改 `put()` 函数，确保在对链表进行操作时加锁。

```

static
void put(int key, int value)
{
    int i = key % NBUCKET;
    // is the key already present?
    struct entry *e = 0;
    for (e = table[i]; e != 0; e = e->next) {
        if (e->key == key)
            break;
    }
    if(e){
        // update the existing key.
        e->value = value;
    } else {
        // the new is new.
        pthread_mutex_lock(&locks[i]);
        insert(key, value, &table[i], table[i]);
        pthread_mutex_unlock(&locks[i]);
    }
}

```

完成修改后依次测试命令，观察到不再有键值丢失的情况。

```

● bruce@Jarvis:~/xv6-labs-2021$ make ph
gcc -o ph -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/ph.c -pthread
● bruce@Jarvis:~/xv6-labs-2021$ ./ph 1
100000 puts, 4.638 seconds, 21559 puts/second
0: 0 keys missing
100000 gets, 4.506 seconds, 22195 gets/second
● bruce@Jarvis:~/xv6-labs-2021$ ./ph 2
100000 puts, 1.964 seconds, 50923 puts/second
0: 0 keys missing
1: 0 keys missing
200000 gets, 3.791 seconds, 52752 gets/second

```

进行 `./grade-lab-thread ph_safe` 和 `./grade-lab-thread ph_fast` 单项测试，观察到测试通过。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-thread ph_safe
make: 'kernel/kernel' is up to date.
== Test ph_safe == make: 'ph' is up to date.
ph_safe: OK (5.8s)
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-thread ph_fast
make: 'kernel/kernel' is up to date.
== Test ph_fast == make: 'ph' is up to date.
ph_fast: OK (13.8s)
```

接下来是进行性能优化，通过增大 NBUCKET，即增加哈希表桶的数目，可以一定程度上减少两个同时运行的线程 `put()` 时对同一个 `bucket` 进行操作的概率，进而减少锁的争用，令 `NBUCKET=10`。

```
#define NBUCKET 10
```

继续运行测试命令，可以看到修改后的性能获得了巨大提升。

```
● bruce@Jarvis:~/xv6-labs-2021$ make ph
gcc -o ph -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/ph.c -pthread
● bruce@Jarvis:~/xv6-labs-2021$ ./ph 1
100000 puts, 2.842 seconds, 35184 puts/second
0: 0 keys missing
100000 gets, 2.858 seconds, 34994 gets/second
● bruce@Jarvis:~/xv6-labs-2021$ ./ph 2
100000 puts, 1.318 seconds, 75867 puts/second
1: 0 keys missing
0: 0 keys missing
200000 gets, 2.606 seconds, 76734 gets/second
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-thread ph_fast
make: 'kernel/kernel' is up to date.
== Test ph_fast == make: 'ph' is up to date.
ph_fast: OK (10.0s)
```

完成实验后，还需要回答一下问题：

Why are there missing keys with 2 threads, but not with 1 thread? Identify a sequence of events with 2 threads that can lead to a key being missing. Submit your sequence with a short explanation in `answers-thread.txt`.

产生该问题的根本原因是在 `ph.c` 的哈希表实现中，`insert()` 函数存在竞态条件。当两个线程同时操作同一个哈希桶时，由于缺少同步机制，会导致新插入的节点覆盖已有节点，造成键丢失。具体过程如下：

线程 A 调用 `put(key1, value1)` 计算出桶索引 `i` 后进入 `insert()` 函数。此时它读取到 `table[i]` 为 `NULL`，但在创建新条目后被操作系统中断。此时线程切换至线程 B，该线程恰好调用 `put(key2, value2)` 且哈希计算也指向同一个桶 `i`。因线程 A 尚未完成操作，线程 B 在 `insert()` 中同样读取到 `table[i]` 为 `NULL`，于是创建新条目并成功插入 `key2`。当线程 A 恢复执行时，它基于先前读取的 `NULL` 值继续操作，将桶头指针指向 `key1` 条目。这个操作直接覆盖了线程 B 已完成的插入结果，导致 `key2` 条目从链表中断开丢失。最终桶 `i` 仅保留 `key1` 条目，而 `key2` 条目虽然被分配内存却未被任何链表引用，造成永久性键缺失。

[实验中遇到的问题和解决方法]

该实验较为简单，没有遇到问题。

[实验心得]

在实验中，我观察到单线程正常的哈希表，多线程时却会因为竞态条件丢失键值，这体现了多线程编程中安全与性能的微妙平衡。这让我明白，并行编程的核心是用好合理同步策略，兼顾安全与效率。本实验提高了我在实际项目中应用并行编程的能力，理解了线程的创建、同步、数据传递和互斥锁的使用。

Barrier

[实验目的]

In this assignment you'll implement a barrier: a point in an application at which all participating threads must wait until all other participating threads reach that point too. You'll use pthread condition variables, which are a sequence coordination technique similar to xv6's sleep and wakeup.

在本实验中，您将实现一个屏障(Barrier)，这是应用程序中的一个同步点，所有参与线程必须在此等待，直至其他所有参与线程也都到达该点。您将使用pthread条件变量，这是一种序列协调技术，其功能类似于xv6中的sleep与wakeup机制。

[实验内容]

在 notxv6/barrier.c 中完成 barrier() 函数。

```
static void
barrier()
{
    // YOUR CODE HERE
    //
    // Block until all threads have called barrier() and
    // then increment bstate.round.
    //
    pthread_mutex_lock(&bstate.barrier_mutex);
    bstate.nthread++;
    if (bstate.nthread == nthread) {
        // 最后一个线程到达时唤醒所有等待的线程并开始新一轮
        bstate.round++;
    }
}
```

```

    bstate.nthread = 0;
    pthread_cond_broadcast(&bstate.barrier_cond); // 唤醒所有等待线程
} else {
    // 等待其他线程到达
    pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
}
pthread_mutex_unlock(&bstate.barrier_mutex);
}

```

函数开始时首先获取互斥锁以保护共享状态的访问。然后将到达 barrier 的线程计数器 `bstate.nthread` 递增，表示又有一个线程到达了同步点。接下来函数需要判断当前线程是否是最后一个到达的线程。

如果当前线程是最后一个到达的线程，那么这个线程就承担起协调者的角色。它会将轮次计数器 `bstate.round` 递增以标记新一轮的开始，同时将线程计数器 `bstate.nthread` 重置为 0，为下一轮 barrier 调用做准备。然后唤醒所有在等待的线程，通知它们可以继续执行了。反之，如果当前线程不是最后一个到达的线程，它就需要等待其他线程。

当线程从等待中被唤醒或者作为最后一个线程完成协调工作后，函数最后会释放互斥锁，允许其他操作继续进行。

完成函数后运行 `make barrier` 首先构建 barrier 程序，然后运行测试命令 `./barrier <threadNum>`:

```

● bruce@Jarvis:~/xv6-labs-2021$ ./barrier 2
OK; passed
● bruce@Jarvis:~/xv6-labs-2021$ ./barrier 5
OK; passed
● bruce@Jarvis:~/xv6-labs-2021$ ./barrier 10
OK; passed

```

然后进行 `./grade-lab-thread barrier` 单项测试

```

● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-thread barrier
make: 'kernel/kernel' is up to date.
== Test barrier == make: 'barrier' is up to date.
barrier: OK (3.4s)

```

[实验中遇到的问题和解决方法]

本次实验不是很复杂，在理解好阻塞线程概念，阅读实验指导并查看该文件其他代码后，就可以很顺利的完成。

[实验心得]

通过实现屏障机制，我对多线程协作中的同步点有了更具体的认识，也切实掌握了条件变量与互斥锁的配合使用。屏障让所有线程在特定点等待，直到全部到达再共同推进，这种设计直观展现了多线程协调的核心思路。实际编码过程中，

对共享状态的保护和线程唤醒时机的把握，让我对同步机制的实际应用有了更深的体会，不再局限于理论层面的认知。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。
然后执行命令 `./grade-lab-thread`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-thread
make: 'kernel/kernel' is up to date.
== Test uthread == uthread: OK (1.6s)
== Test answers-thread.txt == answers-thread.txt: OK
== Test ph_safe == make: 'ph' is up to date.
ph_safe: OK (3.9s)
== Test ph_fast == make: 'ph' is up to date.
ph_fast: OK (10.3s)
== Test barrier == make: 'barrier' is up to date.
barrier: OK (3.3s)
== Test time ==
time: OK
Score: 60/60
```

最后在当前分支（`thread`）下，依次执行以下命令，提交分支到 github：

```
$ git checkout thread #确保在 thread 分支
$ make clean          #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: Multithreading."
$ git push github thread:thread
```

Lab: networking

实验开始前，依次执行以下命令，切换到 net 分支

```
$ git fetch
$ git checkout net
$ make clean
```

Your Job

[实验目的]

Your job is to complete `e1000_transmit()` and `e1000_recv()`, both in `kernel/e1000.c`, so that the driver can transmit and receive packets. You are done when `make grade` says your solution passes all the tests.

您的任务是完善 `kernel/e1000.c` 中的 `e1000_transmit()` 与 `e1000_recv()` 函数，实现网络数据包的发送与接收功能。当 `make grade` 命令显示所有测试用例通过时，即完成本实验。

[实验内容]

按照实验指导要求所述，我们需要在 `kernel/e1000.c` 中完成两个函数的编写 `e1000_transmit` 和 `e1000_recv` 函数。

`e1000_transmit` 函数用于将以太网帧传递给 `e1000` 网络设备的发送描述符环，并启动发送过程，其实现如下：

```
int
e1000_transmit(struct mbuf *m)
{
    acquire(&e1000_lock); // 获取锁
    // 获取当前 TX 环的索引
    uint32_tdt = regs[E1000_TDT];
    // 检查环是否已满 - 如果 DD 位没有设置，说明之前的传输还没完成
    if((tx_ring[tdt].status & E1000_TXD_STAT_DD) == 0) {
        // 环已满，返回错误
        release(&e1000_lock); // 释放锁
        return -1;
    }
    // 释放之前在这个描述符位置传输的 mbuf（如果有的话）
    if(tx_mbufs[tdt] != 0) {
```

```

    mbuffree(tx_mbufs[tdt]);
}
// 填充描述符
tx_ring[tdt].addr = (uint64)m->head; // 数据包内容的地址
tx_ring[tdt].length = m->len;        // 数据包长度
tx_ring[tdt].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS; // 设置必要的命令标志
// 保存 mbuf 指针，以便稍后释放
tx_mbufs[tdt] = m;
// 更新环位置
regs[E1000_TDT] = (tdt + 1) % TX_RING_SIZE;
release(&e1000_lock); // 释放锁
return 0;
}

```

`e1000_transmit(struct mbuf *m)` 函数的作用是将一个网络数据包 `m` 发送到 E1000 网卡设备。函数通过 `acquire(&e1000_lock)` 获取一个全局锁，防止多个处理器或线程同时修改网卡发送环（Transmit Ring），确保并发安全。接着，通过读取寄存器 `E1000_TDT` 获取当前应当写入的新描述符索引 `tdt`。

在进行实际发送之前，函数检查 `tx_ring[tdt]` 的 `status` 字段的 `DD`（Descriptor Done）位，判断该描述符是否已经被网卡硬件处理完毕。如果 `DD` 位尚未设置，说明之前的数据还在处理，发送环当前位置尚未空出，函数便会释放锁并返回 `-1`，表示发送失败。

若当前描述符可用，函数继续操作。若之前在该位置绑定了一个 `mbuf` 缓存，需要先调用 `mbufree()` 释放对应内存，避免内存泄漏。随后，函数使用当前的 `mbuf` 填充描述符，设置数据地址 `addr`、长度 `length`，并在 `cmd` 字段中设置 `EOP` 与 `RS` 位，指示该数据包结束并要求网卡更新 `DD` 位以供下一次发送检查。

为了在未来能够正确释放该 `mbuf`，函数将其指针保存在 `tx_mbufs[tdt]` 中。完成所有设置后，将 `E1000_TDT` 指针向前推进一个位置（使用模 `TX_RING_SIZE` 以实现环形结构），表示新数据已准备就绪，可供网卡硬件读取。

最后，函数释放全局锁表示发送任务成功挂入传输队列。

`e1000_recv` 函数用于接收来自 `e1000` 网络设备的数据包，并将它们传递给网络栈进行处理。

```

static void
e1000_recv(void)
{
    // 计算下一个待处理的接收包索引
    uint32 rdt = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
    // 处理所有可用的包
    while(1) {
        // 检查是否有新包可用，如果没有新包，停止处理
    }
}

```



```

if((rx_ring[rdt].status & E1000_RXD_STAT_DD) == 0) {
    break;
}
// 检查数据包的长度是否超过了 mbuf 的最大大小
if (rx_ring[rdt].length > MBUF_SIZE) {
    panic("MBUF_SIZE OVERFLOW!");
}
// 获取当前 mbuf
struct mbuf *m = rx_mbufs[rdt];
// 更新 mbuf 的长度为描述符中报告的长度
m->len = rx_ring[rdt].length;
// 将 mbuf 传递给网络栈
net_rx(m);
// 分配新的 mbuf 来替换刚刚传递的那个
struct mbuf *new_m = mbufalloc(0);
if(new_m == 0) {
    // 检查内存分配是否失败
    panic("e1000_recv: mbufalloc failed");
}
// 将新 mbuf 的数据指针编程到描述符中
rx_ring[rdt].addr = (uint64)new_m->head;
rx_ring[rdt].status = 0; // 清除状态位
// 保存新的 mbuf 指针
rx_mbufs[rdt] = new_m;
// 移动到下一个描述符
rdt = (rdt + 1) % RX_RING_SIZE;
}
// 更新 RDT 寄存器为最后处理的描述符索引
if(rdt == 0) {
    regs[E1000_RDT] = RX_RING_SIZE - 1;
} else {
    regs[E1000_RDT] = rdt - 1;
}
}
}

```

`e1000_recv(void)` 函数是 E1000 网卡驱动中用于接收数据包的核心例程。它通过轮询方式从接收环形缓冲区 (Receive Ring) 中提取已完成的接收描述符，将网络数据交由上层网络协议栈处理，并为环形缓冲区补充新的缓冲区，以保证接收流程的连续性。

函数首先计算出当前应该处理的接收描述符索引。E1000 网卡通过 `E1000_RDT` 寄存器记录最近一个由驱动处理完的描述符索引，因此通过加一并对环大小取模，可以定位到下一个待处理的接收槽位。接着，函数进入循环，不断检查当前描述符的 `status` 字段是否设置了 DD (Descriptor Done) 位。如果没

有设置，说明网卡还没有将新的数据包写入该位置，函数便停止处理。

当一个接收描述符标记为完成后，驱动首先检查网卡报告的接收数据长度是否超过了 mbuf 能承载的最大值 MBUF_SIZE，以防数据溢出。如果长度合法，驱动从 rx_mbufs 中取出对应的 mbuf 缓冲区，并更新其 len 字段为接收到的数据长度，随后调用 net_rx() 将该数据包交由网络协议栈进一步处理。

数据包处理完毕后，原有的 mbuf 不再使用，驱动需要分配一个新的 mbuf，用于未来接收新的数据包。若分配成功，驱动将新 mbuf 的数据地址写入接收描述符的 addr 字段，并清空 status 字段，表示此位置已准备好接收下一批数据，同时也将新 mbuf 的指针记录在 rx_mbufs 中。

最后，在退出循环之后，驱动更新 E1000_RDT 寄存器，使网卡知道哪些描述符已被处理完毕并已腾空，网卡可以在这些位置写入新的接收数据。

该实验需要两个终端来验证结果，首先在第一个终端内输入 make server

```
○ bruce@Jarvis:~/xv6-labs-2021$ make server
python3 server.py 26099
listening on localhost port 26099
```

然后再第二个终端中输入 make qemu，然后进行 nettests 测试，观察实验结果：

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ nettests
nettests running on port 26099
testing ping: OK
testing single-process pings: OK
testing multi-process pings: OK
testing DNS
DNS arecord for pdos.csail.mit.edu. is 128.52.129.126
DNS OK
all tests passed.
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
```

[实验中遇到的问题和解决方法]

本次实验与计算机网络知识密切相关，在本学期学习了计算机网络课程后，本次实验就不是很难理解。

[实验心得]

通过本次实验，我深入理解了网络驱动中数据包的发送与接收流程，特别是 E1000 网卡的描述符环机制。在实现 e1000_transmit() 和 e1000_recv() 的过程中，我学习了如何与硬件寄存器进行交互，如何使用 mbuf 管理内存，以及如何确保数据传输的正确性与效率。此外，我也更加体会到内核编程中资源管理的重要性，例如合理使用锁、及时释放内存等。整体而言，这是一次理论联系实践的

宝贵机会，巩固了我对操作系统和网络协议栈底层机制的理解。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。

然后执行命令 `./grade-lab-net`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-net
make: 'kernel/kernel' is up to date.
== Test running nettests == (2.2s)
== Test  nettest: ping ==
nettest: ping: OK
== Test  nettest: single process ==
nettest: single process: OK
== Test  nettest: multi-process ==
nettest: multi-process: OK
== Test  nettest: DNS ==
nettest: DNS: OK
== Test time ==
time: OK
Score: 100/100
```

最后在当前分支（`net`）下，依次执行以下命令，提交分支到 `github`:

```
$ git checkout net      #确保在 net 分支
$ make clean            #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: networking."
$ git push github net:net
```

Lab: locks

实验开始前，依次执行以下命令，切换到 lock 分支

```
$ git fetch
$ git checkout lock
$ make clean
```

Memory allocator

[实验目的]

Your job is to implement per-CPU freelists, and stealing when a CPU's free list is empty. You must give all of your locks names that start with "kmem". That is, you should call `initlock` for each of your locks, and pass a name that starts with "kmem". Run `kalloctest` to see if your implementation has reduced lock contention. To check that it can still allocate all of memory, run `usertests sbrkmuch`. Your output will look similar to that shown below, with much-reduced contention in total on kmem locks, although the specific numbers will differ. Make sure all tests in `usertests` pass. `make grade` should say that the `kalloctests` pass.

你的任务是实现每 CPU 空闲列表（per-CPU freelists），并在某个 CPU 的空闲列表为空时执行窃取（stealing）操作。你必须为所有锁命名以"kmem"开头。也就是说，你需要为每个锁调用 `initlock`，并传入以"kmem"开头的名称。运行 `kalloctest` 以检查你的实现是否减少了锁争用。为确保仍能分配全部内存，请运行 `usertests sbrkmuch`。你的输出结果应类似于下方所示——kmem 锁的总争用大幅降低（具体数值可能不同）。确保 `usertests` 所有测试通过。执行 `make grade` 应显示 `kalloctests` 通过。

[实验内容]

原始 xv6 内存分配器使用单一全局空闲链表，由单个锁保护，在多核 CPU 环境中，所有 CPU 在分配/释放内存时必须竞争同一把锁，在高并发场景会导致严重锁竞争，无法充分利用多核并行性。因此，需要为每一个 CPU 都创建一个空闲链表，同时需要解决内存分配公平性与可用性问题。

首先在 `kernel/kalloc.c` 中将 `kmem` 修改为数组，为每个 CPU 分配一个 `kmem`。

```
struct {
```

```

struct spinlock lock;
struct run *freelist;
} kmem[NCPU];

```

修改 kinit 函数，为每个 CPU 的 freelist 分配所有的空闲内存空间。

```

void
kinit()
{
    for(int i = 0; i < NCPU; i++) {
        char name[16];
        snprintf(name, sizeof(name), "kmem_%d", i);
        initlock(&kmem[i].lock, name);
    }
    freerange(end, (void*)PHYSTOP);
}

```

修改 kfree 将释放的物理页加入到当前 CPU 的空闲链表。为了 cpuid() 能够正确获得当前 CPU 编号，需要使用 push_off() 和 pop_off() 来关闭和打开中断，避免进程调度到另一个 CPU 核心。

```

void
kfree(void *pa)
{
    struct run *r;
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");
    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);
    r = (struct run*)pa;
    push_off();
    int cpu = cpuid();
    acquire(&kmem[cpu].lock);
    r->next = kmem[cpu].freelist;
    kmem[cpu].freelist = r;
    release(&kmem[cpu].lock);
    pop_off();
}

```

修改 kalloc 函数，用于查找当前 CPU 空闲链表中是否有空闲块，没有则需要从其他 CPU 中窃取。

```

void *
kalloc(void)
{
    struct run *r;
    push_off();
    int cpu = cpuid();
    acquire(&kmem[cpu].lock);

```

```

r = kmem[cpu].freelist;
if(r)
    kmem[cpu].freelist = r->next;
release(&kmem[cpu].lock);
// 如果当前 CPU 空闲链表为空，尝试从其他 CPU 窃取内存空闲页
if(!r) {
    for(int i = 0; i < NCPU; i++) {
        if(i == cpu) continue;
        acquire(&kmem[i].lock);
        r = kmem[i].freelist;
        if(r) {
            kmem[i].freelist = r->next;
            release(&kmem[i].lock);
            break;
        }
        release(&kmem[i].lock);
    }
}
pop_off();
if(r)
    memset((char*)r, 5, PGSIZE); // fill with junk
return (void*)r;
}

```

在 xv6 的 qemu 环境以下中分别运行 kalloc test、usertests sbrkmuch 和 usertests，观察到所有测试通过。

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ kalloc test
start test1
test1 results:
--- lock kmem/bcache stats
lock: bcache: #test-and-set 0 #acquire() 1248
--- top 5 contended locks:
lock: proc: #test-and-set 47815 #acquire() 338325
lock: proc: #test-and-set 38068 #acquire() 338297
lock: proc: #test-and-set 36410 #acquire() 338297
lock: proc: #test-and-set 35346 #acquire() 338297
lock: proc: #test-and-set 34991 #acquire() 338299
tot= 0
test1 OK
start test2
total free number of pages: 32499 (out of 32768)
.....
test2 OK

```

```

$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED

```

```

test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED

```

[实验中遇到的问题解决方法]

本次实验在实现每 CPU 空闲链表和窃取机制过程中，主要遇到的问题是在开始时，我在 kfree 和 kalloc 中调用 cpuid() 获取当前 CPU ID 时，未加

`push_off()` 封锁中断，造成少数情况下 CPU ID 读取不一致，进而导致链表写入越界或空指针错误。通过补充 `push_off()` 和 `pop_off()` 保证临界区的 CPU 不被切换，最终保证了多核并发下的内存分配安全性，最终运行测试全部通过。

[实验心得]

通过本次实现基于多核优化的内存分配器，我深入理解了 xv6 中 `spinlock` 的使用机制，以及多核系统中如何通过局部性优化减少锁竞争瓶颈。原始设计中所有 CPU 共用一个全局 `freelist`，带来极高的锁争用，而在每 CPU 分配独立链表后，大多数分配与释放操作都可以在本地完成，在资源不足时跨 CPU 窃取，大幅提升并发性能。

这个实验不仅锻炼了我的并发编程能力，也为后续理解更复杂的多核内存管理机制打下了良好基础。

Buffer cache

[实验目的]

Modify the block cache so that the number of acquire loop iterations for all locks in the bcache is close to zero when running `bcachetest`. Ideally the sum of the counts for all locks involved in the block cache should be zero, but it's OK if the sum is less than 500. Modify `bget` and `brelse` so that concurrent lookups and releases for different blocks that are in the bcache are unlikely to conflict on locks (e.g., don't all have to wait for `bcache.lock`). You must maintain the invariant that at most one copy of each block is cached. When you are done, your output should be similar to that shown below (though not identical). Make sure `usertests` still passes. `make grade` should pass all tests when you are done.

修改块缓存，以便在运行 `bcachetest` 时，`bcache` 中所有锁的获取循环迭代次数接近于零。理想情况下，块缓存中涉及的所有锁的计数总和应为零，但如果总和小于 500 也是可以接受的。修改 `bget` 和 `brelse`，使得对 `bcache` 中不同块的并发查找和释放不太可能在锁上发生冲突（例如，不必都等待 `bcache.lock`）。您必须保持不变量，即每个块在缓存中最多只有一份副本。完成之后，您的输出应类似于下面所示（尽管不完全相同）。请确保 `usertests` 仍然可以通过。完成时，`make grade` 应通过所有测试。

[实验内容]

根据实验指导书，实验需要将原本管理缓存块的双向链表移除，采用哈希表来管理，使得每个 bucket 有一个锁而非只有一个 bcache 的全局锁，这样就可以减少锁的争用。

首先需要在 kernel/bio.c 中定义哈希表大小和哈希函数，然后像结构体 bcache 中添加哈希表的定义。

```
#define NBUCKET 13
#define HASH(blockno) (blockno % NBUCKET) // 哈希函数
struct {
    struct spinlock lock; // 用于保护驱逐操作
    struct buf buf[NBUF];
    // 哈希桶
    struct {
        struct spinlock lock;
        struct buf head;
    } bucket[NBUCKET];
} bcache;
```

修改 kernel/buf.h 中 struct buf, 向其中加入时间戳字段用于后续分配算法。

```
uint timestamp; // 最后使用时间戳
```

修改 kernel/bio.c 中的 binit() 函数，加入相关初始化逻辑。

```
void
binit(void)
{
    struct buf *b;
    // 初始化每个哈希桶的锁
    for(int i = 0; i < NBUCKET; i++) {
        initlock(&bcache.bucket[i].lock, "bcache.bucket");
        bcache.bucket[i].head.prev = &bcache.bucket[i].head;
        bcache.bucket[i].head.next = &bcache.bucket[i].head;
    }
    for(b = bcache.buf; b < bcache.buf+NBUF; b++){
        b->next = bcache.bucket[0].head.next;
        b->prev = &bcache.bucket[0].head;
        initsleeplock(&b->lock, "buffer");
        bcache.bucket[0].head.next->prev = b;
        bcache.bucket[0].head.next = b;
        b->timestamp = 0;
        b->dev = 0;
        b->blockno = 0;
        b->refcnt = 0;
    }
}
```



```
}
```

继续修改 `bget` 函数，对于键值对(`dev`, `blockno`)求出哈希值，在对应桶中查找是否已经缓冲。若缓冲，则返回相应地址；没有则需要分配新的缓冲区，从所有桶中找出最久未使用的空闲缓冲区作为目标，将该缓冲区移入哈希值对应桶（本就在该桶中则不需要移动），然后做相应修改后返回对应地址。

```
static struct buf*
bget(uint dev, uint blockno)
{
    struct buf *b;
    int idx = HASH(blockno);
    acquire(&bcache.bucket[idx].lock);
    // Is the block already cached?
    for(b = bcache.bucket[idx].head.next; b != &bcache.bucket[idx].head; b = b->next){
        if(b->dev == dev && b->blockno == blockno){
            b->refcnt++;
            release(&bcache.bucket[idx].lock);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&bcache.bucket[idx].lock);
    acquire(&bcache.lock);
    // 再次检查，避免竞争条件
    acquire(&bcache.bucket[idx].lock);
    for(b = bcache.bucket[idx].head.next; b != &bcache.bucket[idx].head; b = b->next){
        if(b->dev == dev && b->blockno == blockno){
            b->refcnt++;
            release(&bcache.bucket[idx].lock);
            release(&bcache.lock);
            acquiresleep(&b->lock);
            return b;
        }
    }
    release(&bcache.bucket[idx].lock);
    // 查找最久未使用的缓冲区进行驱逐
    struct buf *victim = 0;
    uint min_time = __UINT32_MAX__;
    int victim_idx = -1;
    for(int i = 0; i < NBUCKET; i++) {
        // 最多同时持有两个桶的锁：曾经最优桶和当前最优桶
        acquire(&bcache.bucket[i].lock);
        for(b = bcache.bucket[i].head.next; b != &bcache.bucket[i].head; b = b->next) {
            // 找到新的缓冲块
```

```

    if(b->refcnt == 0 && b->timestamp < min_time) {
        // 更新时，需要解开曾经最优桶的锁（不是本身时），否则可能存在两个以上的桶的锁
        if(victim_idx != -1 && i != victim_idx)
            release(&bcache.bucket[victim_idx].lock);
        min_time = b->timestamp;
        victim = b;
        victim_idx = i;
    }
}

// 如果 i 桶不符合要求，释放其锁
if(i != victim_idx)
    release(&bcache.bucket[i].lock);
}

// 当没有剩余缓冲区可以使用时释放全局锁
// 此时对于每个 i, i != victim_idx 都成立，不会持有任何桶的锁
if(victim == 0) {
    release(&bcache.lock);
    panic("bget: no buffers");
}

if (victim_idx != idx) {
    victim->prev->next = victim->next;
    victim->next->prev = victim->prev;
    release(&bcache.bucket[victim_idx].lock);
    acquire(&bcache.bucket[idx].lock);
    victim->next = bcache.bucket[idx].head->next;
    victim->prev = &bcache.bucket[idx].head;
    bcache.bucket[idx].head->next->prev = victim;
    bcache.bucket[idx].head->next = victim;
}

// 更新 victim 的信息
victim->dev = dev;
victim->blockno = blockno;
victim->valid = 0;
victim->refcnt = 1;
release(&bcache.bucket[idx].lock);
release(&bcache.lock);
acquiresleep(&victim->lock);
return victim;
}

```

继续修改 brelse、bpin、bunpin 函数。

```

void
brelse(struct buf *b)
{
    if(!holdingsleep(&b->lock))

```

```

        panic("brelse");
    releasesleep(&b->lock);
    int idx = HASH(b->blockno);
    acquire(&bcache.bucket[idx].lock);
    b->refcnt--;
    if (b->refcnt == 0)
        b->timestamp = ticks;
    release(&bcache.bucket[idx].lock);
}

void
bpin(struct buf *b) {
    int idx = HASH(b->blockno);
    acquire(&bcache.bucket[idx].lock);
    b->refcnt++;
    release(&bcache.bucket[idx].lock);
}

void
bunpin(struct buf *b) {
    int idx = HASH(b->blockno);
    acquire(&bcache.bucket[idx].lock);
    b->refcnt--;
    release(&bcache.bucket[idx].lock);
}

```

在 xv6 的 qemu 环境以下中分别运行 bcachetest 和 usertests，观察到所有测试通过：

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ bcachetest
start test0
test0 results:
--- lock kmem/bcache stats
lock: bcache.bucket: #test-and-set 0 #acquire() 2191
lock: bcache.bucket: #test-and-set 0 #acquire() 4211
lock: bcache.bucket: #test-and-set 0 #acquire() 2352
lock: bcache.bucket: #test-and-set 0 #acquire() 4348
lock: bcache.bucket: #test-and-set 0 #acquire() 4481
lock: bcache.bucket: #test-and-set 0 #acquire() 6397
lock: bcache.bucket: #test-and-set 0 #acquire() 6678
lock: bcache.bucket: #test-and-set 6 #acquire() 7435
lock: bcache.bucket: #test-and-set 0 #acquire() 6255
lock: bcache.bucket: #test-and-set 0 #acquire() 6265
lock: bcache.bucket: #test-and-set 0 #acquire() 6257
lock: bcache.bucket: #test-and-set 0 #acquire() 4197
lock: bcache.bucket: #test-and-set 0 #acquire() 4197
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 96753 #acquire() 1068
lock: proc: #test-and-set 42631 #acquire() 285350
lock: proc: #test-and-set 39951 #acquire() 285351
lock: proc: #test-and-set 39574 #acquire() 264819
lock: proc: #test-and-set 38106 #acquire() 285350
tot= 6
test0: OK
start test1
test1 OK

test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED

```

[实验中遇到的问题和解决方法]

在该实验中，我主要遇到了两个问题，首先是在查找最久未使用的缓冲区进行驱逐时，即在以下代码片段：

```
for(int i = 0; i < NBUCKET; i++) {
    acquire(&bcache.bucket[i].lock);
    for(b = bcache.bucket[i].head.next; b != &bcache.bucket[i].head; b = b->next) {
        if(b->refcnt == 0 && b->timestamp < min_time) {
            // 忘记的片段
            if(victim_idx != -1 && i != victim_idx)
                release(&bcache.bucket[victim_idx].lock);
            min_time = b->timestamp;
            victim = b;
            victim_idx = i;
        }
    }
    if(i != victim_idx)
        release(&bcache.bucket[i].lock);
}
```

忘记解开曾经最优桶的锁，造成了死锁问题，这个问题很好解决，只要分析好每一个临界资源的获取和释放，很容易发现。

第二个问题则困扰了我很久，修复完死锁问题后，我的代码执行 `usertests` 测试时，总会在 `test writebig` 时输出以下内容

```
test writebig: panic: balloc: out of blocks
```

开始时我以为是自己代码的错误，调整了很多版之后，依旧没有解决该问题。然后开始思索是不是项目本身的问题，看到测试名和错误信息后，我觉得应该是在测试大文件写时，块不够用，这种可能由于文件系统拥有的块数目分配的太少导致的。我将 `kernel/param.h` 中的 `FSSIZE` 从 1000 调整为 2000，便没有了该问题。

```
#define FSSIZE      2000 // size of file system in blocks
```

找出可能的问题后，我上网求证，发现很多博客都提出了该问题，并且在项目 `github` 仓库中，也有很多人提出了该问题和类似的解决方案。链接：[balloc: out of blocks when compiled with -O3 · Issue #59 · mit-pdos/xv6-riscv](https://github.com/mit-pdos/xv6-riscv/issues/59)

[实验心得]

在本次 `Buffer Cache` 实验中，我深刻体会到多线程并发环境下资源管理的复杂性和挑战性。传统的全局锁机制在面对大量并发访问时，会产生严重的锁竞争，从而拖慢系统性能。本实验的优化目标正是缓解这一瓶颈。通过将全局 `bcache.lock` 拆分为多个桶级锁，并采用哈希表结构管理缓存块，我有效地提高了系统对缓存块的并发访问效率，减少了锁的获取冲突。尤其是在 `bget` 和 `brelse`

函数中，合理地划分临界区并控制锁的获取与释放顺序，是保障数据一致性和避免死锁的关键。

总的来说，本实验不仅锻炼了我操作系统层面处理并发问题的能力，也提升了我对缓存设计、锁粒度控制以及死锁规避策略的理解。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。

然后执行命令 `./grade-lab-lock`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-lock
make: 'kernel/kernel' is up to date.
== Test running kallocetest == (33.0s)
== Test  kallocetest: test1 ==
kallocetest: test1: OK
== Test  kallocetest: test2 ==
kallocetest: test2: OK
== Test kallocetest: sbrkmuch == kallocetest: sbrkmuch: OK (4.4s)
== Test running bcachetest == (5.6s)
== Test  bcachetest: test0 ==
bcachetest: test0: OK
== Test  bcachetest: test1 ==
bcachetest: test1: OK
== Test usertests == usertests: OK (90.1s)
== Test time ==
time: OK
Score: 70/70
```

最后在当前分支（`lock`）下，依次执行以下命令，提交分支到 `github`：

```
$ git checkout lock    #确保在 lock 分支
$ make clean           #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: locks."
$ git push github lock:lock
```

Lab: file system

实验开始前，依次执行以下命令，切换到 fs 分支

```
$ git fetch
$ git checkout fs
$ make clean
```

Large files

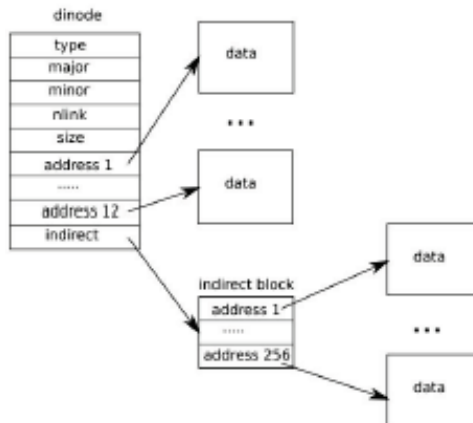
[实验目的]

Modify `bmap()` so that it implements a doubly-indirect block, in addition to direct blocks and a singly-indirect block. You'll have to have only 11 direct blocks, rather than 12, to make room for your new doubly-indirect block; you're not allowed to change the size of an on-disk inode. The first 11 elements of `ip->addrs[]` should be direct blocks; the 12th should be a singly-indirect block (just like the current one); the 13th should be your new doubly-indirect block. You are done with this exercise when `bigfile` writes 65803 blocks and `usertests` runs successfully.

修改 `bmap()` 函数，使其除了直接块和单间接块之外，还能实现双间接块。由于要在 `ip->addrs[]` 中为新的双间接块腾出空间，所以您只能有 11 个直接块，而不是 12 个；您不能更改磁盘上 `inode` 的大小。`ip->addrs[]` 的前 11 个元素应为直接块；第 12 个应为单间接块（与当前的一样）；第 13 个应为您的新双间接块。当 `bigfile` 写入 65803 个块并且 `usertests` 成功运行时，您就完成了此练习。

[实验内容]

与 FAT 文件系统类似，`xv6` 文件系统中的每一个 `inode` 结构体中，采用了混合索引的方式记录数据的所在具体盘块号。每个文件所占用 的前 12 个盘块的盘块号直接记录在 `inode`，即对于任何文件的前 12 KB 数据，都可以通过访问 `inode` 直接得到盘块号。对于大于 12 个盘块的文件，大于 12 个盘块的部分，采用一级索引存储方式存储，可以存储 256 个盘块号。因此，一个文件最大大小为 268 盘块。`Inode` 的数据结构如图所示：



本实验的目的是将一个直接索引节点改为二级间接索引，使一个文件的总大小可以达到 $65803(=256*256+256+11)$ 个。

首先需要将 `kernel/fs.h` 中 `NDIRECT` 从 12 修改为 11，再定义二级页表，修改最大文件，同时定义 `NDBLINDIRECT`，用于表示二级索引块号的总数。

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDBLINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDBLINDIRECT)
```

然后修改 `kernel/file.h` 中 `inode` 结构体和 `kernel/fs.h` 中 `dinode` 结构体，将 `addrs[NDIRECT+1]` 修改为 `addrs[NDIRECT+1]`，包含 11 个直接块+1 个一级间接块+1 个二级间接块。

```
uint addrs[NDIRECT+2];
```

修改 `kernel/fs.c` 中的 `bmap()` 函数，模仿一级索引的实现，在一级索引中再进行一次一级索引的操作。

```
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;
    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;
    if(bn < NINDIRECT){
        // Load indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
```

```

    if((addr = a[bn]) == 0){
        a[bn] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
bn -= NINDIRECT;

// 二级间接块
if(bn < NDBLINDIRECT){
    // 加载二级间接块
    if((addr = ip->addrs[NINDIRECT+1]) == 0)
        ip->addrs[NINDIRECT+1] = addr = balloc(ip->dev);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    // 计算一级间接块索引
    uint idx1 = bn / NINDIRECT;
    uint idx2 = bn % NINDIRECT;
    // 获取或分配一级间接块
    if((addr = a[idx1]) == 0){
        a[idx1] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    // 读取一级间接块并获取数据块地址
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[idx2]) == 0){
        a[idx2] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
panic("bmap: out of range");
}

```

该函数根据块号 (bn) 的范围采用分层处理逻辑：当块号位于直接块范围内时，若检测到 inode 中对应块地址为 0，则调用 `balloc` 分配新物理块并更新 inode 的直接块地址表，随后返回分配的块地址。当块号处于一级间接块范围内时，首先检查间接块指针是否分配，若未分配则创建间接块并更新 inode；接着读取该间接块内容，若目标块地址为 0 则分配新物理块并更新间接块内容，通过 `log_write` 记录写操作后返回块地址。对于二级间接块范围内的块号，先验证

二级间接块指针，若未分配则创建二级间接块并更新 `inode`；读取二级间接块后，根据块号计算一级间接块的索引位置，若该一级间接块未分配则创建并更新二级间接块；随后读取一级间接块内容，再次根据块号计算数据块索引，若数据块未分配则分配物理块并更新一级间接块，最终返回数据块地址。若块号超出所有支持范围，函数触发 `panic` 异常终止执行。

最后修改 `itrunc()` 函数，增加用于释放二级页表数据块的功能：

```
void
itrunc(struct inode *ip)
{
    int i, j;
    struct buf *bp;
    uint *a;
    for(i = 0; i < NDIRECT; i++){
        if(ip->addrs[i]){
            bfree(ip->dev, ip->addrs[i]);
            ip->addrs[i] = 0;
        }
    }
    if(ip->addrs[NDIRECT]){
        bp = bread(ip->dev, ip->addrs[NDIRECT]);
        a = (uint*)bp->data;
        for(j = 0; j < NINDIRECT; j++){
            if(a[j])
                bfree(ip->dev, a[j]);
        }
        brelse(bp);
        bfree(ip->dev, ip->addrs[NDIRECT]);
        ip->addrs[NDIRECT] = 0;
    }
    // 释放二级间接块
    if(ip->addrs[NDIRECT+1]){
        bp = bread(ip->dev, ip->addrs[NDIRECT+1]);
        a = (uint*)bp->data;
        for(i = 0; i < NINDIRECT; i++){
            if(a[i]){
                // 读取每个一级间接块
                struct buf *bp2 = bread(ip->dev, a[i]);
                uint *a2 = (uint*)bp2->data;
                // 释放一级间接块中的所有数据块
                for(j = 0; j < NINDIRECT; j++){
                    if(a2[j])
                        bfree(ip->dev, a2[j]);
                }
            }
        }
    }
}
```

```

        brelse(bp2);
        // 释放一级间接块本身
        bfree(ip->dev, a[i]);
    }
}
brelse(bp);
// 释放二级间接块本身
bfree(ip->dev, ip->addrs[NDIRECT+1]);
ip->addrs[NDIRECT+1] = 0;
}
ip->size = 0;
iupdate(ip);
}

```

通过分层释放机制回收 inode 占用的磁盘空间：首先遍历直接块地址表，对每个非零地址调用 `bfree` 释放物理块并将地址重置为零。接着处理一级间接块，若其指针非空则读取间接块内容，遍历其中所有块地址并释放非零地址对应的物理块，最后释放一级间接块本身并将指针置零。对于二级间接块，若其指针有效则读取二级间接块内容，遍历每个一级间接块指针，当指针非零时，读取对应一级间接块并释放其管理的所有数据块，随后释放该一级间接块本身；完成所有一级间接块处理后，最终释放二级间接块本身并将指针置零。最后将文件大小重置为零并通过 `iupdate` 更新 inode 磁盘信息。

在 xv6 的 qemu 环境以下中分别运行 `bigfile` 和 `usertests`，观察到所有测试通过

```

xv6 kernel is booting

init: starting sh
$ bigfile
.....
.....
.....
.....
.....
wrote 65803 blocks
bigfile done; ok

```

```

test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED

```

[实验中遇到的问题和解决方法]

本次实验再结合操作系统课程文件系统知识上较为简单，代码实现也可以参

考一级索引，并没有遇到问题。

[实验心得]

通过本次实验，我深入体会到文件系统中多级索引结构在支持大文件存储方面的重要作用。相较于理论学习，动手实现双重间接块的过程让我更清楚地理解了 inode 与数据块之间的映射关系，以及如何通过分层索引高效管理大量数据块。同时，我也加深了对 xv6 内核代码结构、缓冲区操作和磁盘块分配机制的掌握。实验整体难度适中，逻辑清晰，增强了我对文件系统设计和实现细节的理解。

Symbolic links

[实验目的]

You will implement the `symlink(char *target, char *path)` system call, which creates a new symbolic link at `path` that refers to file named by `target`. For further information, see the man page `symlink`. To test, add `symlinktest` to the Makefile and run it. Your solution is complete when the tests produce the correct output (including `usertests` succeeding).

您将实现 `symlink(char *target, char *path)` 系统调用，该调用会在 `path` 处创建一个新的符号链接，指向由 `target` 指定的文件。有关更多信息，请参阅 `symlink` 的手册页。要进行测试，请在 `Makefile` 中添加 `symlinktest` 并运行它。当测试产生正确的输出（包括用户测试成功）时，您的解决方案即完成。

[实验内容]

我们需要实现系统调用 `symlink(char *target, char *path)`，首先，需要在 `kernel/syscall.h` 中添加系统调用号

```
#define SYS_symlink 22
```

在 `kernel/syscall.c` 添加系统调用声明

```
extern uint64 sys_symlink(void);
```

```
[SYS_symlink] sys_symlink,
```

在 `user/usys.pl` 中添加用户空间接口

```
entry("symlink");
```

在 user/user.h 用户头文件添加函数声明

```
int symlink(char*, char*);
```

在 kernel/stat.h 添加文件类型

```
#define T_SYMLINK 4 // 符号链接
```

在 kernel/fcntl.h 添加文件标志位

```
#define O_NOFOLLOW 0x004
```

在 kernel/sysfile.c 实现系统调用实现 sys_symlink

```
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;
    begin_op();
    // 创建符号链接文件
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }
    // 将目标路径写入符号链接的数据块
    if(writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)){
        iunlockput(ip);
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}
```

该函数用于在 xv6 操作系统中创建符号链接（symbolic link），函数接受两个字符串参数，target 是符号链接指向的目标路径，path 是新创建的符号链接文件的路径。通过 argstr 函数从用户空间拷贝这两个字符串到内核空间，然后代码调用 begin_op() 开始一个文件系统事务，并使用 create() 函数创建一个类型为 T_SYMLINK 的新 inode，这代表一个符号链接类型的文件。如若创建成功，则通过 writei() 将 target 路径字符串写入到该符号链接 inode 的数据块中，即将目标路径保存在符号链接文件内容中。若写入失败，释放 inode 并终止事务；否则释放 inode 并成功结束事务，返回 0 表示创建成功。

在 kernel/sysfile.c 中修改 sys_open 函数，添加以下代码片段。在创建符号

链接时，如果目标路径是一个符号链接本身，我们需要递归地获取符号链接的目标路径，直到找到一个非符号链接的路径为止； 当搜索次数达到一定次数后表示文件打开失败。

```

. . .
    ilock(ip);
    if(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)){
        // 递归跟踪符号链接，最多 max_depth 层
        for(int depth = 0; depth < max_depth; depth++){
            if(readi(ip, 0, (uint64)path, 0, MAXPATH) <= 0){
                iunlockput(ip);
                end_op();
                return -1;
            }
            iunlockput(ip);
            if((ip = namei(path)) == 0){
                end_op();
                return -1;
            }
            ilock(ip);
            if(ip->type != T_SYMLINK)
                break;
            if(depth == max_depth-1){ // 超过最大深度
                iunlockput(ip);
                end_op();
                return -1;
            }
        }
    }
}

if(ip->type == T_DIR && omode != O_RDONLY){
. . .

```

在 Makefile 文件的 UPROGS 列表添加 \$U/_symlinktest\

在 xv6 的 qemu 环境以下中分别运行 symlinktest 和 usertests, 观察到所有测试通过:

<pre> xv6 kernel is booting init: starting sh \$ symlinktest Start: test symlinks test symlinks: ok Start: test concurrent symlinks test concurrent symlinks: ok </pre>	<pre> test preempt: kill... wait... OK test exitwait: OK test rmdot: OK test fourteen: OK test bigfile: OK test dirfile: OK test iref: OK test forktest: OK test bigdir: OK ALL TESTS PASSED \$ QEMU: Terminated </pre>
--	---

[实验中遇到的问题和解决方法]

在实现符号链接系统调用的过程中，主要遇到的问题集中在递归解析链接路径的逻辑上。开始实现中，对 `sys_open` 的修改只考虑了一次性解析符号链接，导致在符号链接嵌套的场景下不能正确解析最终的目标文件路径。为解决这一问题，我引入了循环结构，通过最多 `max_depth` 次读取与跳转，不断解析链接内容，直到获取一个非符号链接 `inode`。最终解决该问题了。

[实验心得]

通过本次符号链接实验，我对文件系统中的路径解析机制和 `inode` 结构的灵活应用有了更深刻的理解。尤其是递归解析符号链接路径这一过程，使我意识到操作系统需要兼顾功能性与安全性。在实现 `sys_symlink` 时，我也进一步熟悉了 `XV6` 中文件创建、`inode` 分配与数据块写入的底层逻辑。本次实验不仅锻炼了我的系统编程能力，也让我更直观地理解了现实系统中符号链接行为是如何实现的。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。
然后执行命令 `./grade-lab-fs`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-fs
make: 'kernel/kernel' is up to date.
== Test running bigfile == running bigfile: OK (101.2s)
== Test running symlinktest == (0.5s)
== Test  symlinktest: symlinks ==
    symlinktest: symlinks: OK
== Test  symlinktest: concurrent symlinks ==
    symlinktest: concurrent symlinks: OK
== Test usertests == usertests: OK (162.6s)
== Test time ==
time: OK
Score: 100/100
```

最后在当前分支（`fs`）下，依次执行以下命令，提交分支到 `github`：

```
$ git checkout fs      #确保在 fs 分支
$ make clean          #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: file system."
$ git push github fs:fs
```

Lab: mmap

实验开始前，依次执行以下命令，切换到 mmap 分支

```
$ git fetch
$ git checkout mmap
$ make clean
```

mmap

[实验目的]

You should implement enough mmap and munmap functionality to make the mmaptest test program work. If mmaptest doesn't use a mmap feature, you don't need to implement that feature.

您应当实现足够的 mmap 和 munmap 功能，以使 mmaptest 测试程序能够运行。如果 mmaptest 不使用某个 mmap 特性，您则无需实现该特性。

[实验内容]

本次实验是实现 mmap 与 munmap 这两个系统调用，它们可用于在进程之间共享内存，将文件映射到进程地址空间，并作为用户级缺页方案。

在 kernel/syscall.h 添加系统调用号

```
#define SYS_mmap 22
#define SYS_munmap 23
```

在 kernel/syscall.c 添加系统调用声明并在 syscalls 数组中添加相关代码

```
extern uint64 sys_mmap(void);
extern uint64 sys_munmap(void);

[SYS_mmap]    sys_mmap,
[SYS_munmap]  sys_munmap,
```

在 user/usys.pl 添加用户空间系统调用入口

```
entry("mmap");
entry("munmap");
```

在 user/user.h 添加用户空间函数声明

```
void* mmap(void*, int, int, int, int, int);
int munmap(void*, int);
```

要跟踪 mmap 为每个进程映射的内容，首先需要在 kernel/proc.h 定义

VMA 对应的结构体，记录 `mmap` 创建的虚拟内存范围的地址、长度、权限、文件指针等。

```
#define NVMA 16
struct vma {
    uint64 addr;        // 虚拟内存区域的起始地址
    int len;            // 虚拟内存区域的长度
    int prot;           // 内存保护标志
    int flags;          // 映射标志
    int offset;         // 文件偏移量
    struct file* f;     // 指向映射文件的指针
};
```

在结构体 `struct proc` 中添加 VMA

```
struct vma vmas[NVMA];
```

在 `kernel/sysfile.c` 中添加系统调用 `sys_mmap` 和 `sys_munmap` 实现

```
uint64 sys_mmap(void) {
    uint64 addr;
    int len, prot, flags, offset;
    struct file *f;
    struct vma *vma = 0;
    struct proc *p = myproc();
    int i;
    // 解析系统调用参数，出现错误则返回 -1
    if (argaddr(0, &addr) < 0 || argint(1, &len) < 0
        || argint(2, &prot) < 0 || argint(3, &flags) < 0
        || argfd(4, 0, &f) < 0 || argint(5, &offset) < 0) {
        return -1;
    }
    // 只支持 MAP_SHARED 或 MAP_PRIVATE
    if (flags != MAP_SHARED && flags != MAP_PRIVATE) {
        return -1;
    }
    // 若为共享映射 (MAP_SHARED) 且请求写权限，则文件必须可写
    if (flags == MAP_SHARED && f->writable == 0 && (prot & PROT_WRITE)) {
        return -1;
    }
    // 要求偏移量是页大小的整数倍，且长度和偏移非负
    if (len < 0 || offset < 0 || offset % PGSIZE) {
        return -1;
    }
    // 在当前进程中寻找一个空闲的 VMA 结构来保存映射信息
    for (i = 0; i < NVMA; ++i) {
        if (!p->vmas[i].addr) {
            vma = &p->vmas[i];
        }
    }
}
```



```

        break;
    }
}
if (!vma) {
    return -1;
}
// 假设用户传入的 addr 总为 0，由内核选择页对齐的地址
addr = MMAPMINADDR;
for (i = 0; i < NVMA; ++i) {
    if (p->vmas[i].addr) {
        // 找到已映射区域中最大的地址，并在其后分配
        addr = max(addr, p->vmas[i].addr + p->vmas[i].len);
    }
}
addr = PGROUNDUP(addr); // 向上对齐到页边界
// 地址不能超过用户空间上限
if (addr + len > TRAPFRAME) {
    return -1;
}
// 填充 VMA 信息
vma->addr = addr;
vma->len = len;
vma->prot = prot;
vma->flags = flags;
vma->offset = offset;
vma->f = f;
filedup(f); // 增加文件引用计数
return addr;
}

```

函数首先使用 `argaddr`、`argint` 和 `argfd` 等工具函数解析用户传入的各个参数，包括期望的映射地址 `addr`、映射长度 `len`、权限 `prot`（如读/写）、标志 `flags`、文件描述符对应的文件结构 `f`，以及文件偏移量 `offset`。若参数非法（如偏移量非页对齐、长度或偏移为负等），则立即返回 `-1`。

接着，函数检查映射类型是否合法，仅支持 `MAP_SHARED` 和 `MAP_PRIVATE` 两类。如果是 `MAP_SHARED` 并且用户请求写权限时，必须保证对应文件是可写的，否则会拒绝映射。

之后函数在当前进程的 `vmass`（虚拟内存区域数组）中查找空闲条目，以保存本次映射的信息。如果已无空闲槽位，则返回错误。假设用户传入的 `addr` 总是为 `0`，函数自动为映射选择一个不会与现有映射冲突的地址，选取已存在的所有 `VMA` 中最大地址之后的位置作为起始地址，并进行页对齐。若选定地址加上映射长度超出用户空间范围（即 `TRAPFRAME` 位置），也会返回失败。

最后，函数将映射信息填写进选中的 `vma` 结构体中，包括地址、长度、权限、标志、文件指针与偏移等信息，并调用 `filedup(f)` 增加文件引用计数，确保文件在映射期间不会被关闭或回收。

```
uint64 sys_munmap(void) {
    uint64 addr, va;
    int len;
    struct proc *p = myproc();
    struct vma *vma = 0;
    uint maxsz, n, n1;
    int i;
    // 参数解析失败或对齐检查不通过返回 -1
    if (argaddr(0, &addr) < 0 || argint(1, &len) < 0) {
        return -1;
    }
    if (addr % PGSIZE || len < 0) {
        return -1;
    }
    // 查找包含该地址区间的 VMA
    for (i = 0; i < NVMA; ++i) {
        if (p->vmass[i].addr && addr >= p->vmass[i].addr
            && addr + len <= p->vmass[i].addr + p->vmass[i].len) {
            vma = &p->vmass[i];
            break;
        }
    }
    if (!vma) {
        return -1;
    }
    if (len == 0) {
        return 0;
    }
    // 如果是共享映射，需要把被修改的脏页写回文件
    if ((vma->flags & MAP_SHARED)) {
        // 每次最多写入磁盘的大小
        maxsz = ((MAXOPBLOCKS - 1 - 1 - 2) / 2) * BSIZE;
        for (va = addr; va < addr + len; va += PGSIZE) {
            // 跳过非脏页
            if (uvmgetdirty(p->pagetable, va) == 0) {
                continue;
            }
            // 写回该页内容到文件中
            n = min(PGSIZE, addr + len - va);
            for (i = 0; i < n; i += n1) {
                n1 = min(maxsz, n - i);
```

```

    begin_op();
    ilock(vma->f->ip);
    if (writei(vma->f->ip, 1, va + i, va - vma->addr + vma->offset + i, n1) != n1) {
        iunlock(vma->f->ip);
        end_op();
        return -1;
    }
    iunlock(vma->f->ip);
    end_op();
}
}
}

// 实际从页表中解除映射
uvmunmap(p->pagetable, addr, (len - 1) / PGSIZE + 1, 1);
// 更新 VMA 区间状态
if (addr == vma->addr && len == vma->len) {
    vma->addr = 0;
    vma->len = 0;
    vma->offset = 0;
    vma->flags = 0;
    vma->prot = 0;
    fileclose(vma->f);
    vma->f = 0;
} else if (addr == vma->addr) {
    vma->addr += len;
    vma->offset += len;
    vma->len -= len;
} else if (addr + len == vma->addr + vma->len) {
    vma->len -= len;
} else {
    // 中间部分不允许拆分
    panic("unexpected munmap");
}
return 0;
}

```

函数首先解析用户传入的两个参数：`addr` 表示希望解除映射的起始虚拟地址，`len` 表示解除的长度（单位是字节）。若参数解析失败，或者地址不是页对齐、长度为负，则返回错误 `-1`。紧接着，函数遍历当前进程的 `vmass` 表，查找包含 `[addr, addr+len)` 区间的 VMA 条目，确保用户请求的解除区域确实属于某个已映射区域。

若成功匹配 VMA，且解除长度为 0，函数直接返回。否则，如果该映射是共享映射（`MAP_SHARED`），且用户对映射内存有写入，则需要将被修改过的脏

页内容写回到对应文件。为此，函数使用 `uvmgetdirty` 判断每一页是否被修改，如果是脏页则使用 `writei` 结合文件偏移进行写回，并使用 `begin_op` 和 `end_op` 包裹磁盘写操作以确保原子性。

接着，函数调用 `uvmunmap`，将虚拟地址范围从进程页表中解除映射，并设置 `do_free=1` 以释放对应的物理页。最后，函数更新 `vma` 状态。如果解除的区间正好对应整个 `VMA`，则清空该条目，并关闭文件、释放资源。如果解除的是 `VMA` 的前半段或后半段，则通过调整 `addr`、`offset` 和 `len` 保留剩余映射。但如果尝试解除的是 `VMA` 中间一段，系统不支持将 `VMA` 拆分，会触发 `panic("unexpected munmap")` 报错。

在 `kernel/vm.c` 中添加 `uvmgetdirty` 和 `uvmsetdirtywrite` 函数，其中 `uvmgetdirty` 检查某个虚拟地址是否对应“脏页”，`uvmsetdirtywrite` 函数用来设置某虚拟页为“脏页”并赋予写权限

```
int uvmgetdirty(pagetable_t pagetable, uint64 va) {
    pte_t *pte = walk(pagetable, va, 0);
    if(pte == 0) {
        return 0;
    }
    return (*pte & PTE_D);
}
// 设置虚拟地址 va 所对应页表项 (PTE) 的 dirty 标志和写标志
int uvmsetdirtywrite(pagetable_t pagetable, uint64 va) {
    pte_t *pte = walk(pagetable, va, 0);
    if(pte == 0) {
        return -1;
    }
    *pte |= PTE_D | PTE_W;
    return 0;
}
```

修改 `kernel/proc.c` 中 `exit` 函数，在 `exit` 函数中增加部分代码，在进程退出时，需要像 `munmap()` 一样对文件映射部分内存进行取消映射。

```
.....
if(p == initproc)
    panic("init exiting");
int i;
struct vma* vma;
uint maxsz = ((MAXOPBLOCKS - 1 - 1 - 2) / 2) * BSIZE;
uint64 va;
uint n, n1, r;
for (i = 0; i < NVMA; ++i) {
    if (p->vmass[i].addr == 0) {
```

```

        continue;
    }
    vma = &p->vmass[i];
    if ((vma->flags & MAP_SHARED)) {
        for (va = vma->addr; va < vma->addr + vma->len; va += PGSIZE) {
            if (uvmgetdirty(p->pagetable, va) == 0) {
                continue;
            }
            n = min(PGSIZE, vma->addr + vma->len - va);          for (r = 0; r < n; r += n1) {
                n1 = min(maxsz, n - i);
                begin_op();
                ilock(vma->f->ip);
                if (writei(vma->f->ip, 1, va + i, va - vma->addr + vma->offset + i, n1) != n1)
            {
                iunlock(vma->f->ip);
                end_op();
                panic("exit: writei failed");
            }
            iunlock(vma->f->ip);
            end_op();
        }
    }
}
uvmunmap(p->pagetable, vma->addr, (vma->len - 1) / PGSIZE + 1, 1);
vma->addr = 0;
vma->len = 0;
vma->offset = 0;
vma->flags = 0;
vma->offset = 0;
fclose(vma->f);
vma->f = 0;
}
// Close all open files.
for(int fd = 0; fd < NOFILE; fd++){
    if(p->ofile[fd]){
        struct file *f = p->ofile[fd];
        fclose(f);
        p->ofile[fd] = 0;
    }
}
}
. . . . .

```

修改 kernel/proc.c 中 exit 函数，在 fork() 函数中增加部分代码，使得使用 fork() 创建子进程时，同时将父进程的 VMA 结构体进行拷贝，从而获得相同的

文件映射内存。

```
...
np->cwd = idup(p->cwd);
for (i = 0; i < NVMA; ++i) {
    if (p->vmass[i].addr) {
        np->vmass[i] = p->vmass[i];
        filedup(np->vmass[i].f);
    }
}
safestrncpy(np->name, p->name, sizeof(p->name));
...
```

最后再修改 kernel/vm.c 中 uvmunmap 函数，避免不合法的 panic

```
if((*pte & PTE_V) == 0){
    continue;
}
if(PTE_FLAGS(*pte) == PTE_V){
    continue;
}
```

完成编写后，在 Makefile 文件中 UPROGS 添加一行 \$U/_mmaptest\ 配置，然后进行程序测试。

在 xv6 的 qemu 环境中运行 mmaptest 和 usertests，观察到测试通过。

```
$ mmaptest
mmap_test starting
test mmap f: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test mmap two files
test mmap two files: OK
mmap_test: ALL OK
fork_test starting
fork_test OK
mmaptest: all tests succeeded

test exitiput: OK
test iput: OK
test mem: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

[实验中遇到的问题和解决方法]

在本次实现 mmap 和 munmap 的过程中，最初遇到的问题主要集中在三个方面：一是对用户传入参数的校验不充分，导致 mmaptest 中部分映射操作因非法偏移量或地址越界而失败；二是 VMA 地址分配策略初始实现不够健壮，导致多个 mmap 操作分配到了重叠区域，进而引发页表覆盖；三是在进程 exit() 时忘记处理共享映射区域的脏页写回问题，造成数据未同步到文件。为了解决这些问题，我首先增加了对偏移量页对齐、地址对齐、边界合法性的严格检查，其次

重新设计了 VMA 分配逻辑，从现有最大地址向上页对齐分配，最后在 `exit()` 函数中完整模拟 `munmap` 行为，遍历所有 VMA 写回脏页并释放映射内存，确保文件内容与进程退出状态一致。通过以上修正，`mmaptest` 顺利通过。

[实验心得]

本次 `mmap/munmap` 实验是整个 `xv6` 实验体系中最贴近现代操作系统机制的一部分，不仅涉及虚拟内存管理，还融合了文件系统、页表操作、系统调用与进程生命周期等多个模块的协作。在实验过程中，我不仅需要为每个进程维护虚拟内存区域（VMA）元信息，还需处理用户权限校验、文件页缓存的写回、`fork` 过程中的内存继承等细节。

本次实验使我对 `xv6` 的用户层调用、内核态实现、内存与文件管理等多个子系统有了系统化的认识，也让我体会到一个简单但功能完整的内核，如何以高度模块化方式将复杂机制协同起来，为用户提供高效稳定的服务接口。

提交

首先创建一个 `time.txt` 文件，在其中存放一个整数，记录实验的小时数。然后执行命令 `./grade-lab-mmap`，获取该 Lab 的实验测试结果。

```
● bruce@Jarvis:~/xv6-labs-2021$ ./grade-lab-mmap
make: 'kernel/kernel' is up to date.
== Test running mmaptest == (2.0s)
== Test  mmaptest: mmap f ==
mmaptest: mmap f: OK
== Test  mmaptest: mmap private ==
mmaptest: mmap private: OK
== Test  mmaptest: mmap read-only ==
mmaptest: mmap read-only: OK
== Test  mmaptest: mmap read/write ==
mmaptest: mmap read/write: OK
== Test  mmaptest: mmap dirty ==
mmaptest: mmap dirty: OK
== Test  mmaptest: not-mapped unmap ==
mmaptest: not-mapped unmap: OK
== Test  mmaptest: two files ==
mmaptest: two files: OK
== Test  mmaptest: fork_test ==
mmaptest: fork_test: OK
== Test usertests == usertests: OK (87.5s)
== Test time ==
time: OK
Score: 140/140
```

最后在当前分支（`mmap`）下，依次执行以下命令，提交分支到 `github`：

```
$ git checkout mmap      #确保在 mmap 分支
$ make clean             #清理构建的中间文件
$ git add .
$ git commit -m "Finish Lab: mmap."
$ git push github mmap:mmap
```